Agent Communication Protocols:
MCP, A2A, and Google ADK

Building Multi-Agent Systems with MCP & Google ADK

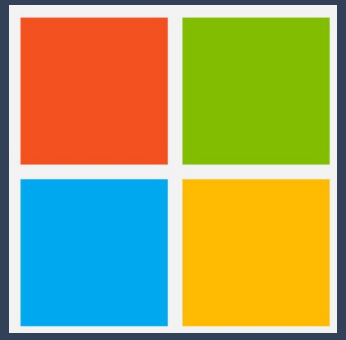
From naive chatbots to production-ready negotiation agents

{ik} | INTERVIEW KICKSTART

Introduction - Jeevendra Singh

- I have been working as a Software Engineer since 2015
- I've worked for companies like Ericsson, SAP, Q2, and now Microsoft
- I have been working at Microsoft for 4+ years
- In my career, I've worked as a full-stack developer, integration architect, and now focusing on Agentic AI development
- Built intelligent API platforms and copilots using Generative AI, RAG, and Agent Orchestration
- Trained over 2000+ learners on Agentic AI, Full-stack, DSA and System Design







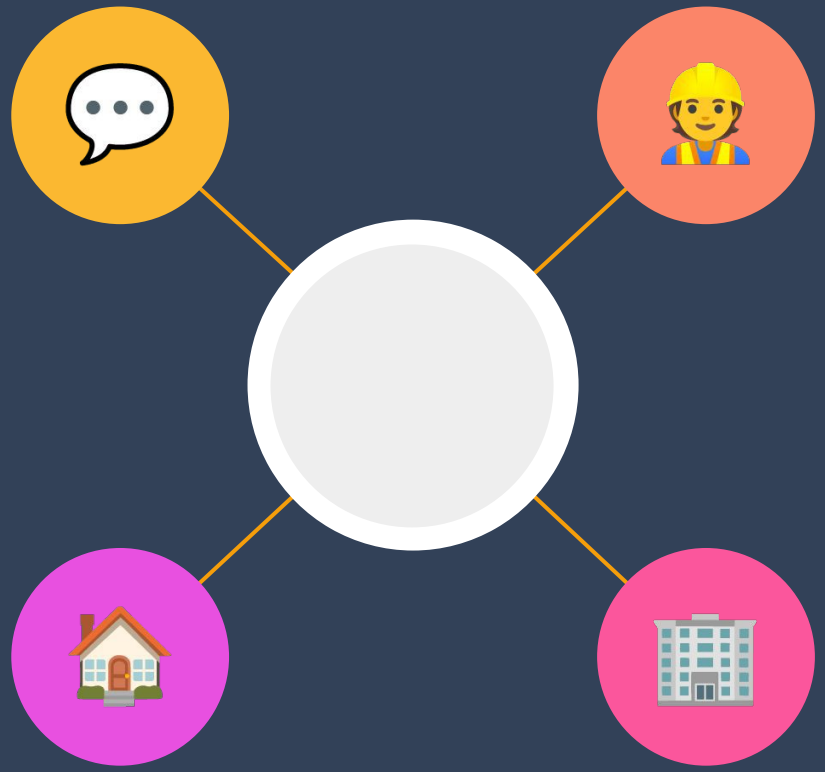




{ik} INTERVIEW KICKSTART

Welcome to today's class, before we begin, pop into the chat:

Hannah Chen



Data Scientist

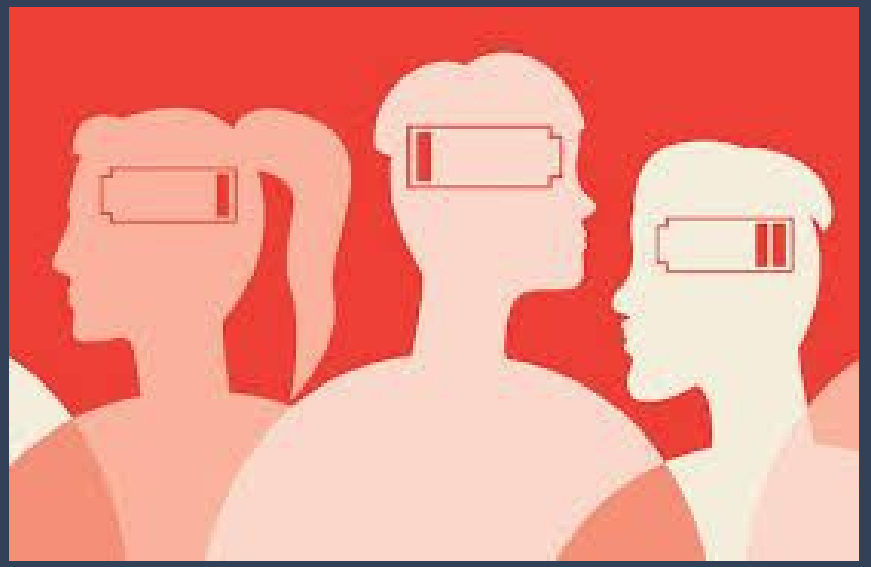
Bay Area

Grammarly, ex-Uber

{ik} INTERVIEW KICKSTART

Structure of Class

1. **Sunday Class - 4 hr (Sunday)**
2. **Assignments -** MCQ & Coding Assignment

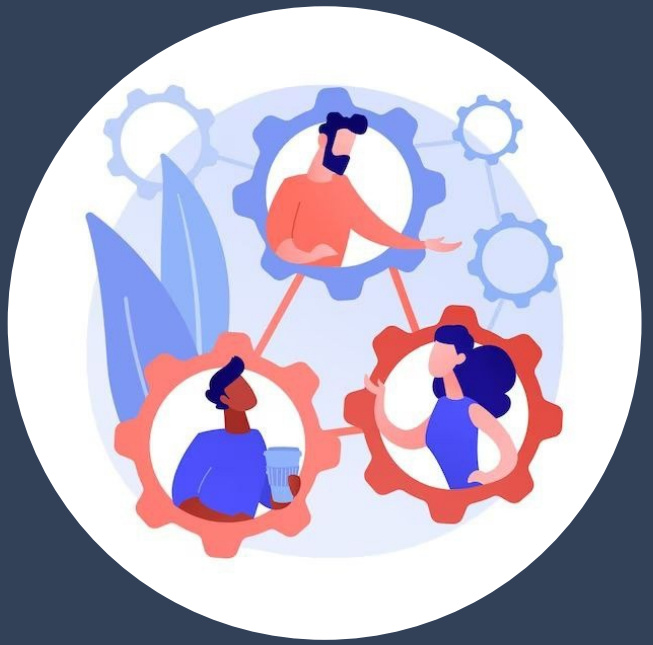




{ik} | INTERVIEW KICKSTART

Optimize Your Experience

- ✅ Interact with your instructors via Sunday live class.
- ✅ Don't be shy to speak up and get clarifications !





- ✅ Solve your assignments, MCQs and other assessments and get feedback (Thursday review session)
- ✅ Use your resources! It's your experience – what you put in is what you'll get out.

{ik} | INTERVIEW KICKSTART

Don't worry!
We are here to help at your every step !
But, how???



{ik} | INTERVIEW KICKSTART

Are you using all these IK support features?

- **Uplevel** - contains all the learning resources - videos, MCQs, assignments
- **Optional** Post class videos for additional reference
- **Technical coaching sessions** on Wednesday to clarify all your queries
- **Discord** community for peer learning
- **TAs** to help answer the queries in the live class
- **Support tickets** - Raise them any time you want any clarifications / tech concerns, etc on UpLevel

{ik} | INTERVIEW KICKSTART

Success Hacks

- Be prepared for the class by watching the pre class videos
- Attend every class
- Participate in class
- Attempt all the assignments
- Actively advocate for yourself
- Keep your goal in mind
- Consistency is the key

Be patient with yourself !

{ik} | INTERVIEW KICKSTART

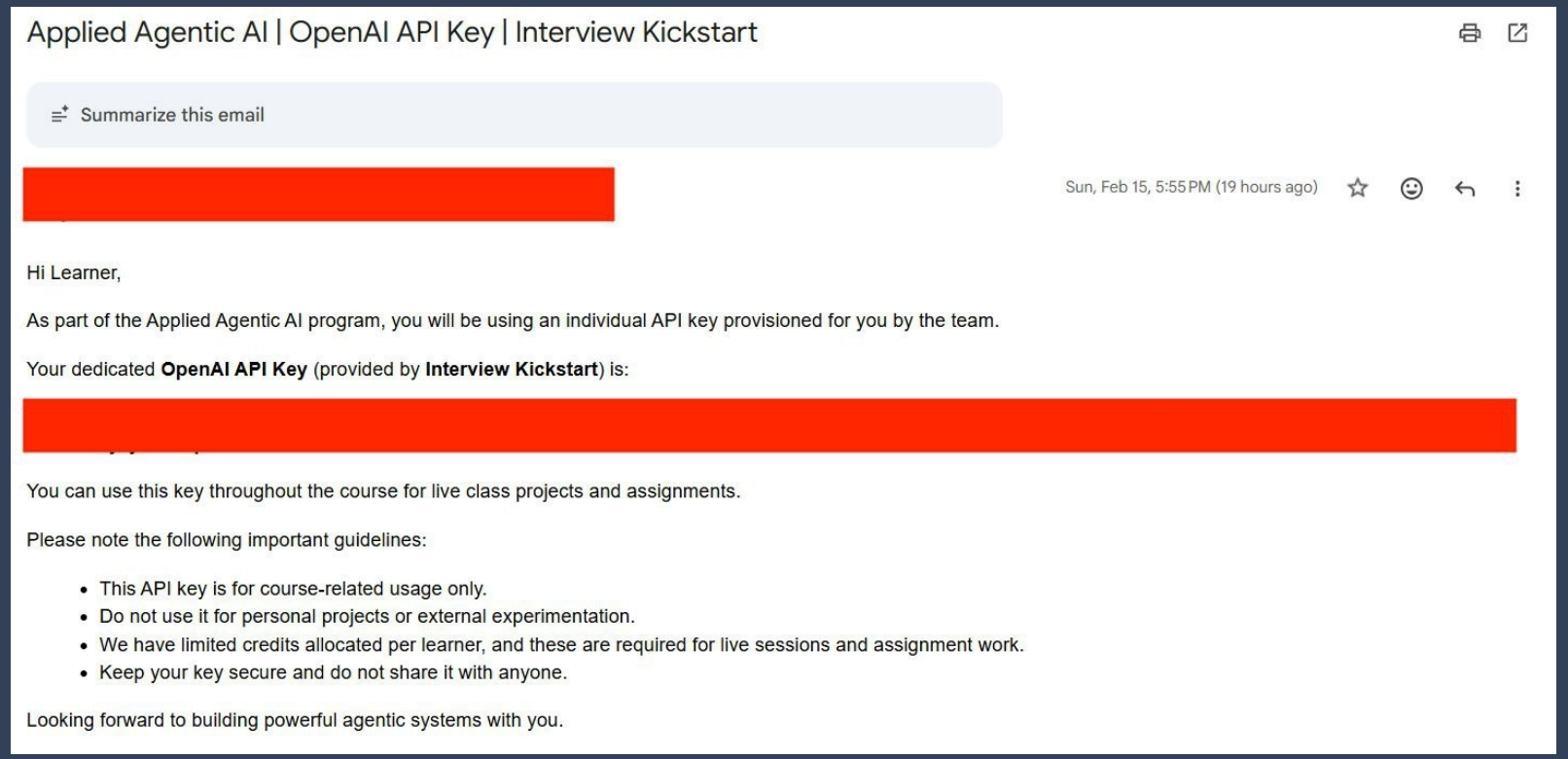
Check Your Email for API Key

You should have received an email like the one shown below containing your OpenAI API key.

If you do not see this email:

- Please check your inbox
- Also check your spam or junk folder

If you still cannot find it, please reach out to the Operations Team operations@interviewkickstart.com



Applied Agentic AI | OpenAI API Key | Interview Kickstart

Summarize this email

[redacted] Sun, Feb 15, 5:55 PM (19 hours ago)

Hi Learner,

As part of the Applied Agentic AI program, you will be using an individual API key provisioned for you by the team.

Your dedicated **OpenAI API Key** (provided by **Interview Kickstart**) is:

[redacted]

You can use this key throughout the course for live class projects and assignments.

Please note the following important guidelines:

- This API key is for course-related usage only.
- Do not use it for personal projects or external experimentation.
- We have limited credits allocated per learner, and these are required for live sessions and assignment work.
- Keep your key secure and do not share it with anyone.

Looking forward to building powerful agentic systems with you.

{ik} INTERVIEW KICKSTART

Clone & Practice: Your Learning Repository



real-estate-negotiation-simulator

https://github.com/jeev1992/real-estate-negotiation-simulator

What's Included:



2 Complete Modules

Module 1: MCP · Module 2: Google ADK + A2A



Working Code Examples

Ready-to-run demos, practice exercises, and solutions for each module

Getting Started

```
git clone https://github.com/jeev1992/real-estate-negotiation-simulator
```

1. Install dependencies: pip install -r requirements.txt
2. Copy .env.example to .env and add your OpenAI API key (GitHub PAT for the GitHub demo)
3. Run demos from each module directory

{ik} | INTERVIEW KICKSTART

How to Follow Along

Setup & Running the Demos

```
# Clone and set up
git clone https://github.com/jeev1992/real-estate-negotiation-simulator && cd real-estate-negotiation-simulator
python -m venv .venv && .venv\Scripts\activate          # Windows (macOS/Linux: source .venv/bin/activate)
pip install -r requirements.txt
cp .env.example .env                                    # add OPENAI_API_KEY (and GITHUB_TOKEN for the GitHub demo)

# Module 1 -- MCP
python m1_mcp/demos/01_initialize_handshake.py
python m1_mcp/demos/02_tool_loop_trace.py
python m1_mcp/github_agent_client.py "Find popular MCP server implementations"
python m1_mcp/sse_agent_client.py "What is 742 Evergreen Terrace worth?"   # after starting the servers with --sse

# Module 2 -- ADK + A2A
adk web m2_adk_multiagents/adk_demos/                   # 9 concept demos in one dropdown
adk web --a2a m2_adk_multiagents/negotiation_agents/    # buyer, seller, negotiation + A2A endpoints
python m2_adk_multiagents/a2a_demos/a2a_13_orchestrated_negotiation.py
```

Prerequisites: Python 3.10+, an OpenAI API key, and a GitHub personal access token for the GitHub MCP demo (it uses GitHub's hosted MCP server — no Node.js needed).

First `adk` run asks "Enable telemetry? [Y/n]" — Enter means yes; answer n or run `adk telemetry disable` to opt out. Each module has a README with per-demo instructions.

What exactly are we building?

{ik} | INTERVIEW KICKSTART

Problem statement

In Week 6 we build a **real estate negotiation simulator** where two AI agents — a Buyer and a Seller — autonomously negotiate the purchase of 742 Evergreen Terrace, Austin TX 78701 (4 BR / 3 BA / 2,400 sqft / listed at $485,000). The negotiation runs for at most 5 rounds and must end in a deal or a clean deadlock.

The agents and their constraints

Party	Goal	Starting position	Walk-away
Buyer Agent (gpt-4o)	Buy at lowest price	Offer ~$425,000	Over $460,000
Seller Agent (gpt-4o)	Sell at highest price	Counter $477,000	Below $445,000

The problem we're solving (in the real world)

- A buyer for 742 Evergreen Terrace wants to pay as little as possible (budget: $460K)
- The seller won't go below their mortgage floor ($445K) but listed at $485K
- Both sides need live market data to justify every move — and neither may see the other's private numbers
- The negotiation must terminate within 5 rounds with either a deal or a clear deadlock

Today's Agenda

1



MCP - External Data for Agents

2



Google ADK + A2A Protocol

{ik} | INTERVIEW KICKSTART

MCP: External Data for Agents

MCP Model Context Protocol is a standard that lets agents call external tools without knowing where the data comes from.

```
Agent:  "What tools do you have?"          -> tools/list
Server: "get_market_price, ..."             -> tool schemas (JSON Schema)
Agent:  "Call get_market_price(...)"        -> tools/call
Server: {estimated_value: 462000}           -> structured result
```

By the end of this module:

- You'll understand the MCP protocol (handshake, tools, resources, prompts)
- You'll have seen a live agent use GitHub's official MCP server — tools and a resource
- You'll have built and inspected custom MCP servers
- You'll understand stdio vs SSE vs Streamable HTTP transports — and where auth fits

{ik} | INTERVIEW KICKSTART

API vs CLI vs MCP: Three Paths to External Systems

> "Agents are only as useful as the systems they can reach." — Anthropic

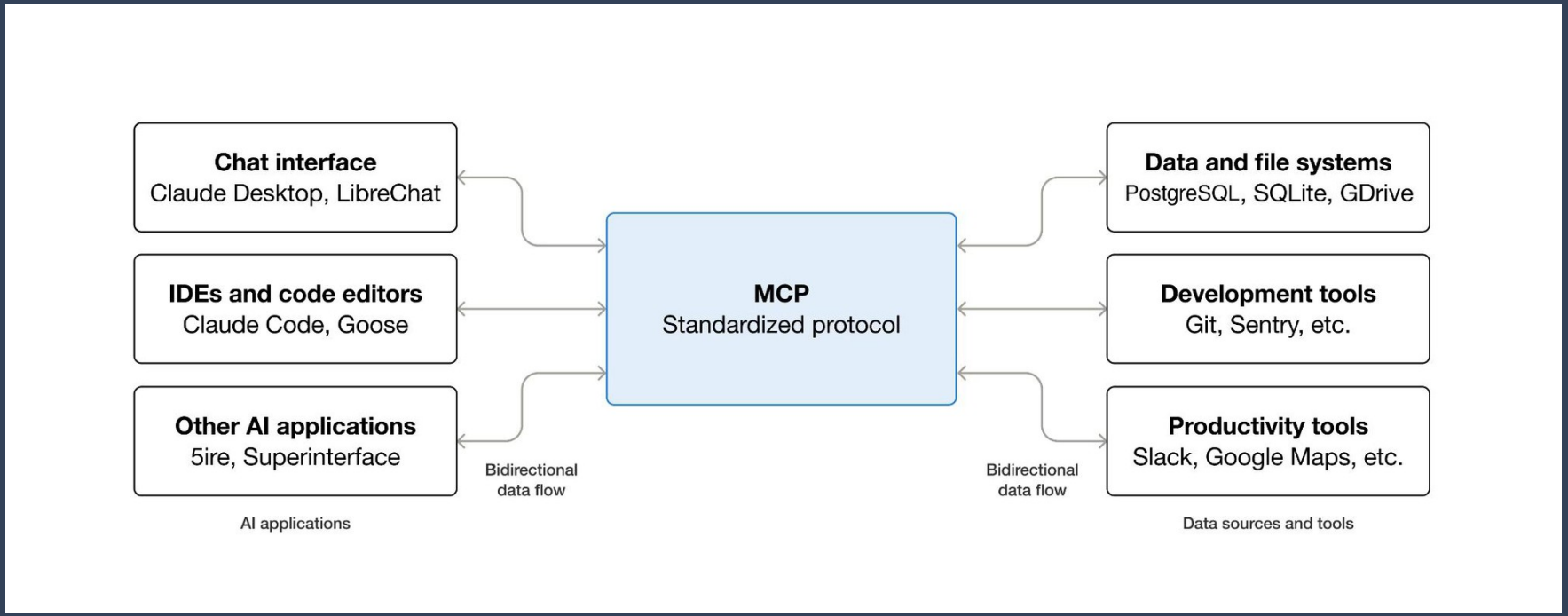
Path	How it works	Limitation
Direct API	Agent calls HTTP endpoints directly	M×N integration problem — each pair is bespoke
CLI	Agent runs shell commands	No reach to web/mobile/cloud — needs a filesystem
MCP	Standard protocol — discovery + invocation + auth	Upfront investment, but portable across all clients

WHY MCP WINS FOR PRODUCTION

- One remote server reaches any compatible client (Claude, ChatGPT, Cursor, VS Code, Google ADK)
- Auth, discovery, and rich semantics are standardized
- Build once, use everywhere — no per-client integration code

{ik} INTERVIEW KICKSTART

What is the Model Context Protocol (MCP)?



- MCP (Model Context Protocol) is an open-source standard for connecting AI applications to external systems.
- Using MCP, AI applications like Claude or ChatGPT can connect to data sources (local files, databases), tools (search engines, calculators) and workflows (specialized prompts).
- Think of MCP like a USB-C port for AI applications: one standard way to plug AI applications into external systems.

{ik} | INTERVIEW KICKSTART

MCP: Core Operations and Benefits

MCP (Model Context Protocol) is a thin RPC layer over **JSON-RPC 2.0** that standardizes three operations between an agent host and an external tool server.

1. **initialize**
   Handshake, version and capability negotiation
2. **tools/list**
   The server advertises typed tool schemas
3. **tools/call**
   The host invokes a tool by name with validated arguments

Everything else — resources, prompts, streaming — is layered on this base.

Why it matters

N + M, not N × M
One pricing server wired once, used by every agent in your stack regardless of language.

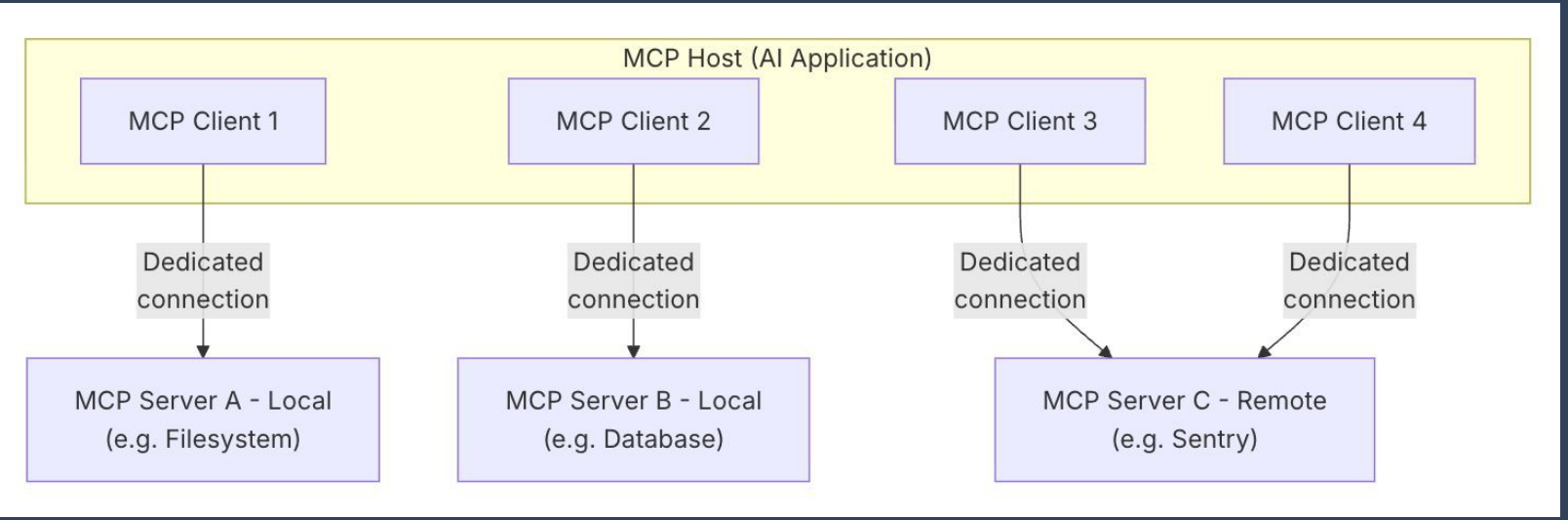
Out-of-process tools
A tool crash does not crash the agent. The server is isolated compute.

Same shape you know
If you know OpenAI function calling or Anthropic tool use, MCP is the same shape — but the registry is reusable and lives out of process.

Participants

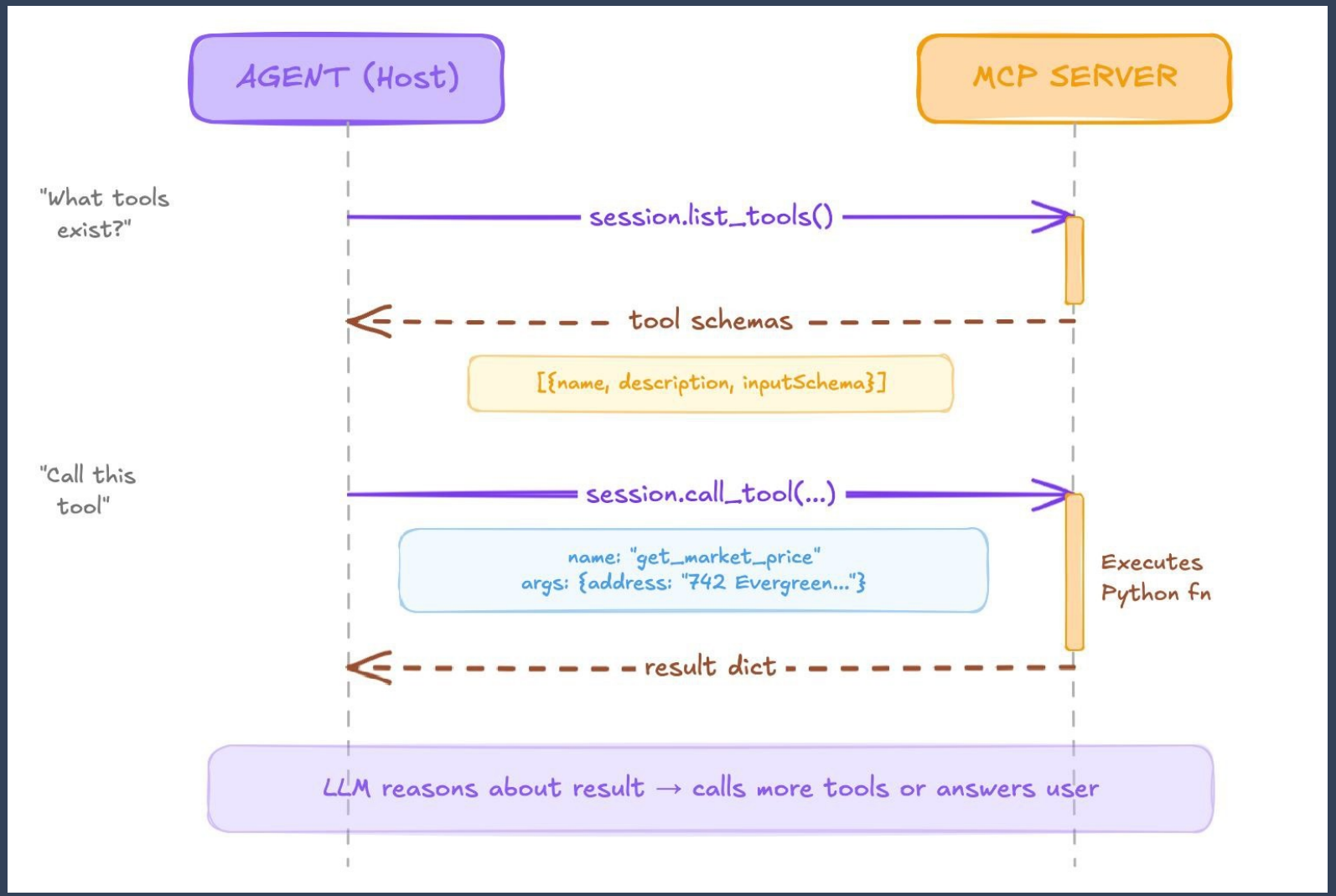
- **MCP Host:** The AI application that coordinates and manages one or multiple MCP clients
- **MCP Client:** A component that maintains a connection to an MCP server and obtains context from it for the host
- **MCP Server:** A program that provides context to MCP clients

For example: VS Code is an MCP host. When it connects to the Sentry MCP server, it creates an MCP client for that connection; when it also connects to the local filesystem server, it creates a second client. One host, one client per server.



{ik} INTERVIEW KICKSTART

MCP: Agent ↔ Server Sequence



{ik} INTERVIEW KICKSTART

MCP Primitives & Content Types: Four Primitives, Three Content Types

Primitive	Direction	Decorator	When to pick it
Tool	Model invokes with arguments	`@mcp.tool()`	Side-effecting action that needs parameters
Resource	Host fetches by URI	`@mcp.resource("scheme://path")`	Read-only document, static or computed
Prompt	User selects a template	`@mcp.prompt("name")`	Parameterized reusable instructions
Sampling	Server asks host to call the LLM	(host-side capability)	Server needs LLM reasoning without holding API keys

Tools are most of MCP usage. Resources and prompts are what the host and the user control.

CONTENT TYPES — WHAT A TOOL CAN RETURN

Type	JSON shape	When to use
TextContent	`{"type": "text", "text": "..."}`	Most common — tool results
ImageContent	`{"type": "image", "data": "base64..."}`	Charts, screenshots
EmbeddedResource	`{"type": "resource", "resource": {...}}`	References to stored data

{ik} | INTERVIEW KICKSTART

Three Transports: stdio, SSE, Streamable HTTP

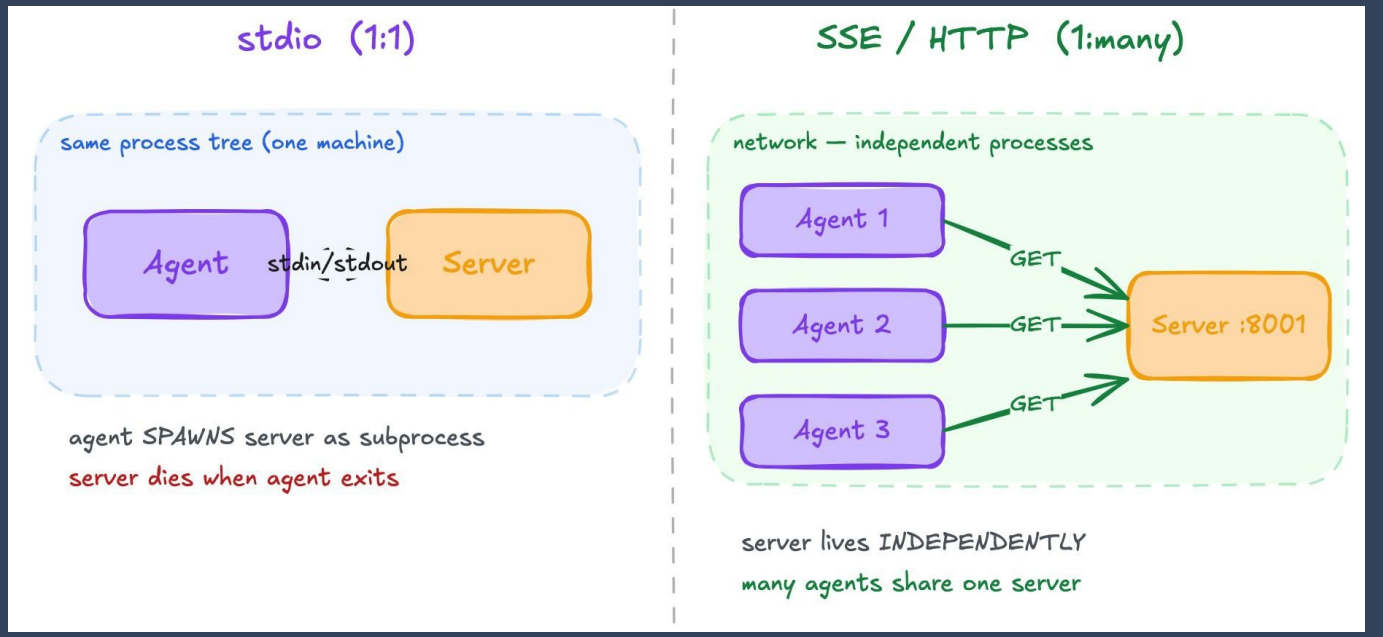
Same protocol, different carrier. Switch with a runtime flag — tool functions change nothing.

Transport	Shape	When to use
stdio	Subprocess pipes — lowest latency	Local dev, integration tests, single-tenant desktop apps. Spawn the server, pipe stdin/stdout, kill on exit.
SSE	Server-sent events over HTTP	Legacy remote deployments. Two endpoints (/sse + /messages). Use only if you inherited it.
Streamable HTTP	One HTTP endpoint; responses can stream	The spec's recommended remote transport. Use for new deployments — and it's where MCP's authorization spec applies.

```
# stdio - subprocess, single tenant
mcp.run()                                    # default

# Streamable HTTP - multi tenant, new standard
mcp.settings.host = "0.0.0.0"
mcp.settings.port = 8001
mcp.run(transport="streamable-http")         # preferred
for remote

# SSE - legacy, avoid for new servers
mcp.run(transport="sse")
```



{ik} INTERVIEW KICKSTART

Demo 01: The MCP Handshake

What Happens on the Wire

```
python m1_mcp/demos/01_initialize_handshake.py      # no API key needed
```

Raw JSON-RPC frames — no SDK, no abstraction:

```
>>> client -> server: initialize
    {"jsonrpc": "2.0", "id": 1, "method": "initialize",
     "params": {"protocolVersion": "2024-11-05", "capabilities": {},
                "clientInfo": {"name": "demo-client", "version": "0.1"}}}
<<< server -> client: initialize result
    {"result": {"protocolVersion": "2024-11-05",
                "capabilities": {"experimental": {}, "prompts": {"listChanged": false},
                                 "resources": {"subscribe": false, "listChanged": false},
                                 "tools": {"listChanged": false}},
                "serverInfo": {"name": "real-estate-pricing", "version": "1.30.0"}}}
>>> notifications/initialized
>>> tools/list
<<< tools/list result -> 2 tools with full JSON schemas
```

5 frames total. After this handshake the client knows what tools exist and can call them. JSON Schema is generated from Python type hints. (serverInfo.version is the installed mcp package version — the SDK reports it when a server doesn't set its own.)

Demo 02: The Tool Loop

Model ↔ Host ↔ Server

```
python m1_mcp/demos/02_tool_loop_trace.py      # requires OPENAI_API_KEY
```

```
[t= 0.00s] HOST    connecting to MCP server (stdio subprocess)
[t= 1.81s] HOST    tools/list -> 2 tools discovered
[t= 2.30s] MODEL   receiving prompt + tool catalog
[t= 4.84s] MODEL   emitted tool_use(get_market_price)
[t= 4.84s] HOST    translating tool_use -> tools/call
[t= 4.84s] SERVER  returned CallToolResult
[t= 4.84s] HOST    injecting tool_result into model context
[t=10.38s] MODEL   emitted final assistant text            # timings vary by machine
```

Three actors, clear roles:

Actor	What it does
Model (gpt-4o-mini)	Decides WHICH tools to call based on the query + schemas
Host (your Python code)	Translates between OpenAI's tool-call format and MCP
Server (pricing_server.py)	Executes the tool and returns structured data

{ik} INTERVIEW KICKSTART

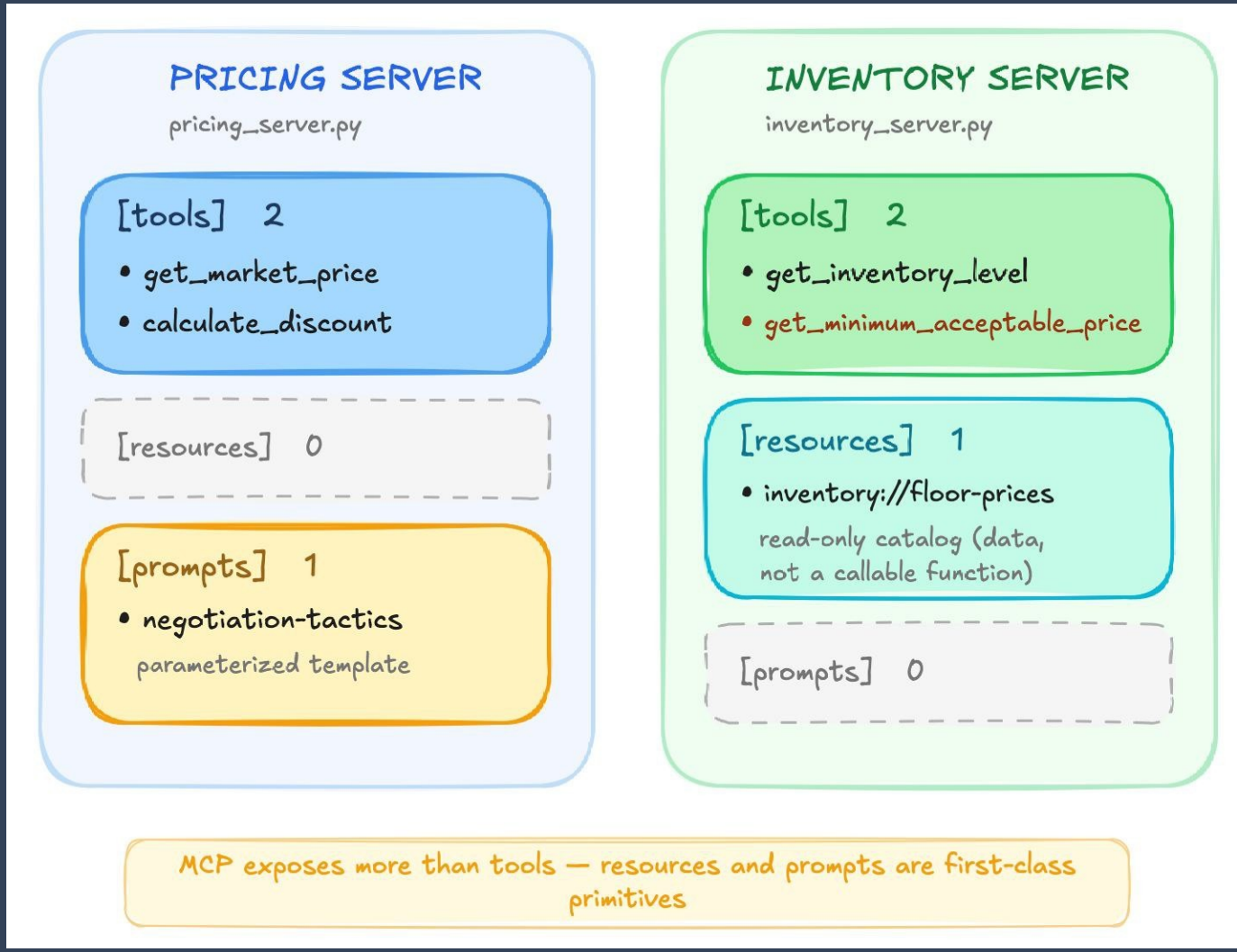
Demo 03: List All Primitives

Tools, Resources, and Prompts

```
python m1_mcp/demos/03_list_all_primitives.py  # no API key
```

What to point out live:

- MCP exposes more than tools; resources and prompts are first-class primitives too
- inventory://floor-prices is readable data, not a callable function
- negotiation-tactics is a prompt template the host can expand into the conversation



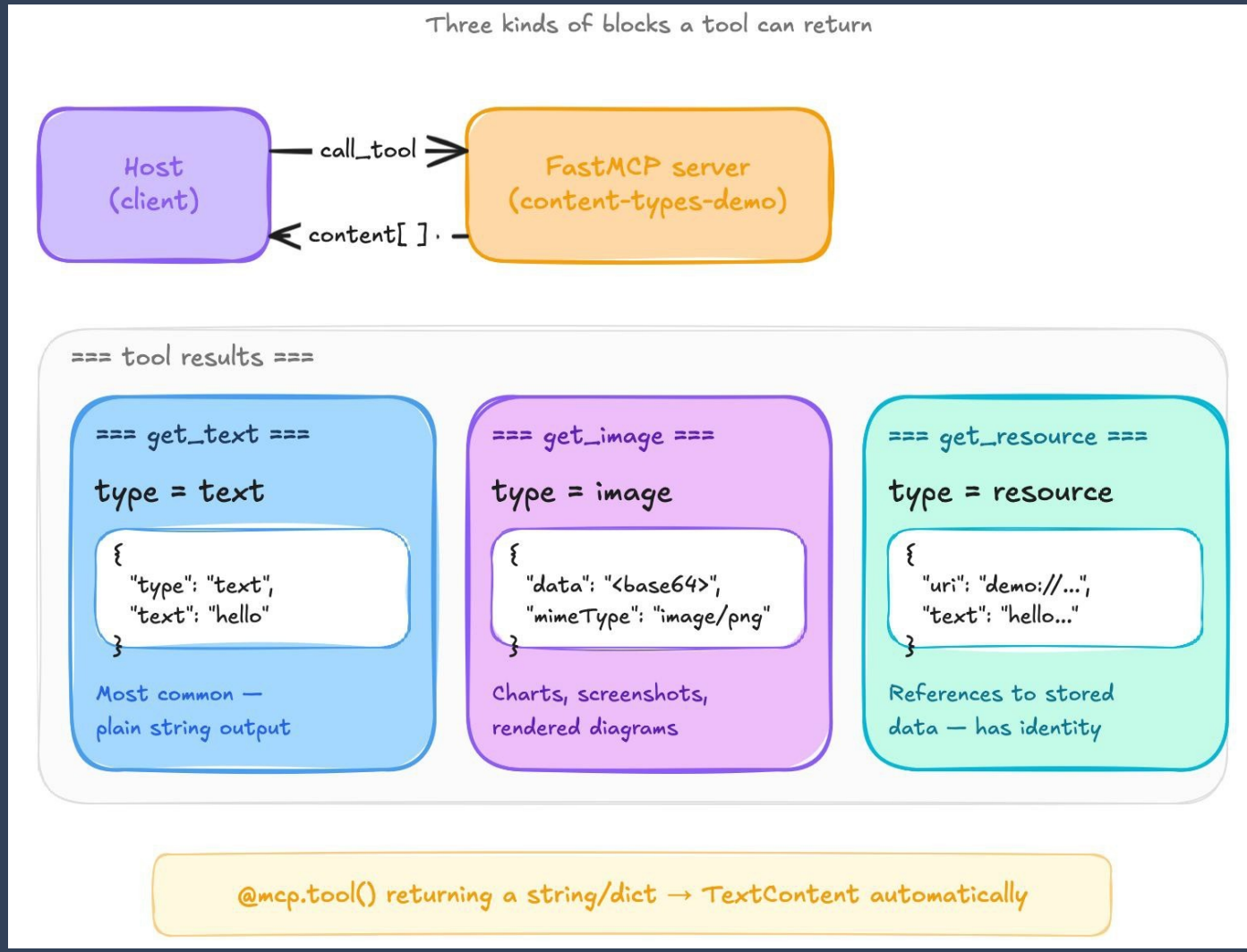
Demo 04: Content Types

Text, Image, and Embedded Resource

```
python m1_mcp/demos/04_content_types.py  # no API key
```

Why it matters:

- Most workshop tools return TextContent, but MCP can also return images and embedded resources
- The protocol standardizes result blocks, so the host can render them consistently
- Demo 04 makes the JSON shape visible before you need to debug it in Module 2



GitHub MCP Agent (Live Demo): An LLM Agent Using GitHub via MCP

```
python m1_mcp/github_agent_client.py "Find popular MCP server implementations"
python m1_mcp/github_agent_client.py --attach repo://modelcontextprotocol/python-sdk/contents/README.md "Summarize what this SDK provides"
```

WHAT HAPPENS

- **Connects to GitHub's hosted MCP server** — https://api.githubcopilot.com/mcp/readonly over Streamable HTTP, with your PAT as a Bearer token (read-only: a class demo can't change anything)
- **Discovers all three primitives:** 29 tools (model-controlled), 5 repo:// resource templates (host-controlled), 2 prompts (user-controlled)
- **With --attach, the host reads a resource** (the README) and puts it in the chat — the model never has to fetch it
- **gpt-4o picks tools, the host calls them via MCP,** results go back into the chat until the model answers

THE AGENTIC LOOP

```
for iteration in range(max_iterations):
    response = await openai.create(messages=messages, tools=openai_tools)
    if response.tool_calls:
        for call in response.tool_calls:
            result = await session.call_tool(call.name, call.args)  # MCP
            messages.append({"role": "tool", "content": result})
    else:
        return response.content  # final answer
```

This is the pattern ADK automates. In Module 2, McpToolset + LlmAgent replace this whole loop with one declaration: tools=[McpToolset(...)].

{ik} | INTERVIEW KICKSTART

Custom MCP Servers

Building Your Own MCP Server

```python
# pricing_server.py - a few lines to expose a tool
from mcp.server.fastmcp import FastMCP
mcp = FastMCP("real-estate-pricing")

@mcp.tool()
def get_market_price(address: str, property_type: str = "single_family") -> dict:
    """Get market price analysis for a property."""
    return {"estimated_value": 462_000, "comparable_sales": [...], ...}
```

WHAT @MCP.TOOL() DOES

- Reads the function signature → JSON Schema (inputSchema)
- Reads the docstring → tool description
- Registers it as a callable MCP tool

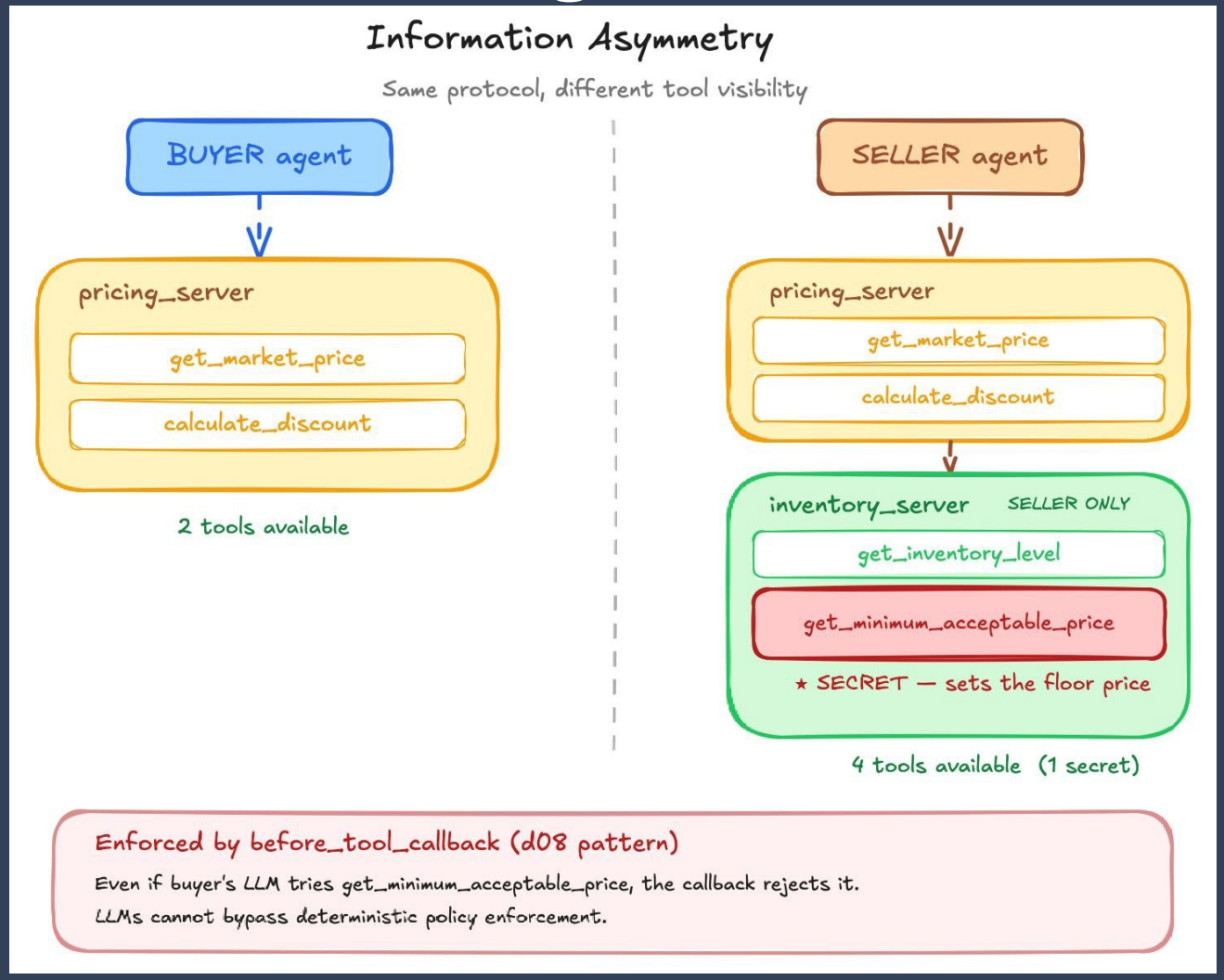
"Group tools around intent, not endpoints." A single get_market_price beats get_comps + get_sqft + get_days_on_market — Anthropic

Server	Tools	Who uses it (Module 2)
pricing_server.py	get_market_price, calculate_discount	Both buyer and seller
inventory_server.py	get_inventory_level, get_minimum_acceptable_price	Seller only

{ik} | INTERVIEW KICKSTART

Information Asymmetry via MCP: Different Agents, Different Tools

The floor price ($445K) lives in the server, not the code.



- get_minimum_acceptable_price() comes from MCP at runtime — only the seller gets it (enforced in Module 2)

Real production systems work this way: same protocol, different tool access per agent.

{ik} INTERVIEW KICKSTART

SSE MCP Agent (Live Demo): Same Loop, HTTP Transport

```
# Terminal 1: start MCP servers in SSE mode
python m1_mcp/pricing_server.py --sse --port 8001
python m1_mcp/inventory_server.py --sse --port 8002
# Terminal 2: run the agent
python m1_mcp/sse_agent_client.py "What is 742 Evergreen Terrace worth?"
# With both servers:
python m1_mcp/sse_agent_client.py --both "What's the seller's minimum price?"
```

	github_agent_client	sse_agent_client
Transport	Streamable HTTP (hosted server)	SSE (HTTP)
Server	GitHub's official MCP server (api.githubcopilot.com)	pricing_server.py + inventory_server.py
Multi-server	No	Yes (--both merges 4 tools from 2 servers)
Model	gpt-4o	gpt-4o-mini
Tool loop	Identical	Identical

The agent loop is identical. Only the connection setup changes. --both reveals information asymmetry: with both servers the agent can see the seller's $445K floor. In Module 2 the buyer's allowlist blocks that tool.

{ik} INTERVIEW KICKSTART

SSE vs stdio vs Streamable HTTP: Three Transports Side by Side

STDIO (DEMOS 01–04)

```
server_params = StdioServerParameters(command="python", args=["pricing_server.py"])
async with stdio_client(server_params) as (read, write):
    # server is a subprocess - dies when you disconnect
```

SSE (sse_agent_client.py)

```
# Terminal 1: start server
python m1_mcp/pricing_server.py --sse --port 8001
# Terminal 2: connect agent
python m1_mcp/sse_agent_client.py "What is 742 Evergreen Terrace worth?"
```

STREAMABLE HTTP (DEMO 05)

```
# Terminal 1: server
python m1_mcp/demos/05_streamable_http_transport.py --serve --port 8765
# Terminal 2: client
python m1_mcp/demos/05_streamable_http_transport.py --client --port 8765
```

The agent code is identical — only the connection setup changes. list_tools() and call_tool() work the same way regardless of transport.

{ik} | INTERVIEW KICKSTART

Demo 05: Streamable HTTP Results

HTTP Transport in Action

```
# Terminal 1:
python m1_mcp/demos/05_streamable_http_transport.py --serve --port 8765
# Terminal 2:
python m1_mcp/demos/05_streamable_http_transport.py --client --port 8765
```

CLIENT OUTPUT

```
Connecting client to http://127.0.0.1:8765/mcp
discovered tools: ['echo']
response: echo: hello over HTTP
```

THE SERVER IS 5 LINES

```
mcp = FastMCP("http-transport-demo")

@mcp.tool()
def echo(text: str) -> str:
    """Echo back the supplied text."""
    return f"echo: {text}"

mcp.run(transport="streamable-http")
```

Same session.list_tools(), same session.call_tool(). Only the import changes. The server is an independent HTTP process — it survives client disconnection, and multiple clients can connect at once.

{ik} | INTERVIEW KICKSTART

What MCP Does Not Solve for You

Idempotency

Retries after timeouts hit your server twice. A hung tool call on the host side does not mean the server did not execute. Make every tool safely retryable, or return an idempotency key.

Cost accounting

MCP doesn't know get_market_price costs money in provider fees. Wire telemetry at the host level and correlate tool name, duration, and downstream cost.

Determinism

Two calls with identical arguments can return different data if the backing system drifts. If reasoning must be reproducible, cache results or pin them to a snapshot.

Prompt injection defense

A tool result containing "ignore previous instructions" still lands in the model's context. Sanitize or structure outputs; don't treat tool results as trusted.

Audit

The server log is not enough. Correlate the host-side JSON-RPC id with the server-side access log, or you can't reconstruct what the model caused.

Use MCP for the protocol shape. Put the rest of your production concerns in the host layer.

{ik} | INTERVIEW KICKSTART

Module 1: What You Learned

Concept	What it means	Where we saw it
MCP protocol	JSON-RPC handshake + tool discovery + invocation	Demo 01 (raw frames)
Three actors	Model (decides), Host (bridges), Server (executes)	Demo 02 (timestamped trace)
Four primitives	Tools, Resources, Prompts, Sampling	Demo 03, github_agent_client.py
Content types	TextContent, ImageContent, EmbeddedResource	Demo 04
Three transports	stdio (1:1), SSE (legacy), Streamable HTTP (remote)	Demos 01–05
Tool design	Group around intent, not endpoints	get_market_price (one call)
Information asymmetry	Different agents see different tools	Buyer vs seller server access
Agentic loop	LLM + tool schemas → decide → call → reason	github_agent_client.py

What carries forward to Module 2: pricing_server.py and inventory_server.py are used directly by the buyer/seller agents · the hand-written loop is replaced by McpToolset + LlmAgent · information asymmetry is enforced by before_tool_callback allowlists (what an agent can call) and include_contents (what it can read).

Module 1 - Implementation Notes: MCP Deep Dive



{ik} | INTERVIEW KICKSTART

No Question Is Too Small Every Question Deserves an Answer!!

Ask about anything:

- Concepts from today's modules
- How this applies to your work
- Implementation details
- Clarifications on what we covered

Quick Request

- Please do not mind if a question gets skipped accidentally
- Repost politely in chat if needed
- We will try our best to cover as many questions as possible

Remember: If you're wondering about it, others probably are too.



Time for a Break



A 10-minute break is starting now. Please return when the timer ends to continue the presentation

{ik} | INTERVIEW KICKSTART

Module 1 → Module 2: What Changes

In Module 1, you wrote the tool loop by hand (~60 lines):

```
tools = await session.list_tools()
openai_tools = mcp_tools_to_openai_functions(tools)
for iteration in range(max_iterations):
    response = await openai.create(messages=messages, tools=openai_tools)
    if response.tool_calls:
        for call in response.tool_calls:
            result = await session.call_tool(call.name, call.args)
            messages.append({"role": "tool", "content": result})
    else:
        return response.content
```

In Module 2, one declaration replaces all of that:

```
root_agent = LlmAgent(
    tools=[McpToolset(connection_params=StdioConnectionParams(
        server_params=StdioServerParameters(command="python", args=["pricing_server.py"])
    ))]
)
```

Module 1 teaches you what's happening inside. Module 2 lets you stop writing it.

{ik} | INTERVIEW KICKSTART

Today's Agenda

1



MCP - External Data for Agents



2



Google ADK + A2A Protocol

{ik} | INTERVIEW KICKSTART

Google ADK + A2A Protocol

The goal: *Build a complete multi-agent negotiation system using Google ADK*

By the end of this module you'll have:

- A buyer agent with MCP pricing tools
- A seller agent with MCP pricing + inventory tools (information asymmetry)
- A negotiation orchestrator that runs multi-round buyer ↔ seller negotiation
- Buyer and seller exposed as A2A network services with Agent Cards (agent.json)

buyer

LlmAgent

+ McpToolset ×1
pricing tools + submit_offer
include_contents= "none"
can't see the floor

seller

LlmAgent

+ McpToolset ×2
pricing + inventory
sees the floor price
+ submit_decision

negotiation

LoopAgent

max_iterations = 5
SequentialAgent
buyer → seller
escalate on ACCEPT

```
Run any of these:
adk web negotiation_agents/          → chat with each agent
adk web --a2a negotiation_agents/    → same + Agent Cards
python a2a_demos/a2a_13_orchestrated_negotiation.py  → full A2A loop
```

{ik} | INTERVIEW KICKSTART

negotiation_agents/: The Heart of Module 2

```
negotiation_agents/
  buyer_agent/agent.py     → LlmAgent + McpToolset (pricing only)
  seller_agent/agent.py    → LlmAgent + McpToolset ×2 (pricing + inventory)
  negotiation/agent.py     → LoopAgent(SequentialAgent(buyer, seller))
```

Run the agents:

```
# Chat with them interactively
adk web m2_adk_multiagents/negotiation_agents/

# Same, plus A2A endpoints + Agent Cards
adk web --a2a m2_adk_multiagents/negotiation_agents/
```

Before we build this, we'll learn each piece through 9 interactive demos. Each demo introduces ONE ADK concept — function tools, MCP tools, sessions, sequential, parallel, loop, agent-as-tool, callbacks, event stream.

{ik} | INTERVIEW KICKSTART

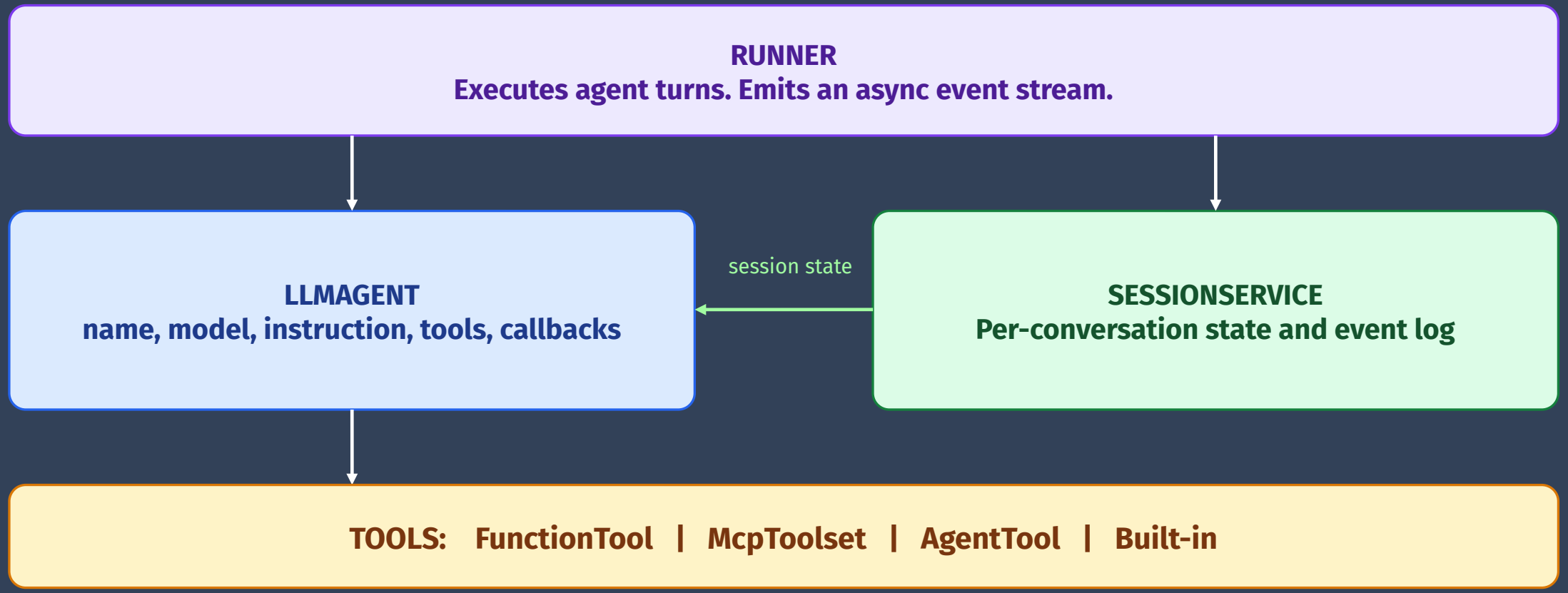
What is Google ADK (Agent Development Kit)

Google's Agent Development Kit (ADK) is an open-source framework for building, testing, evaluating and deploying agents and multi-agent systems. It's designed like real software development — modules, tools, testing, monitoring. It works with any model (here: gpt-4o via LiteLLM) and integrates deeply with Gemini and Vertex AI.

- **Agent:** the core execution unit — goal-driven, calls tools, uses memory, talks to users
- **Tools:** capabilities the agent calls to act (functions, MCP servers, other agents, built-ins)
- **Runner:** runs an agent turn and emits an event stream (adk web, your own scripts, cloud runtimes)
- **Sessions, state & artifacts:** per-conversation, per-user and per-app memory
- **Workflow / orchestration:** Sequential, Parallel and Loop agents — and in ADK 2.x the new graph-based Workflow
- **Callbacks, telemetry & evaluation:** policy hooks, logging, tracing and eval harnesses

This workshop runs on ADK 2.11. Everything here works on 2.x; a few names changed (MCPToolset → McpToolset) and the workflow agents are deprecated in favor of Workflow — slide after Demo 06.

Core Primitives



Four types you touch every day: LlmAgent, Runner, SessionService, and some toolset (usually McpToolset). Everything else is extension.

{ik} INTERVIEW KICKSTART

Demo 01: Basic LlmAgent (Concept)

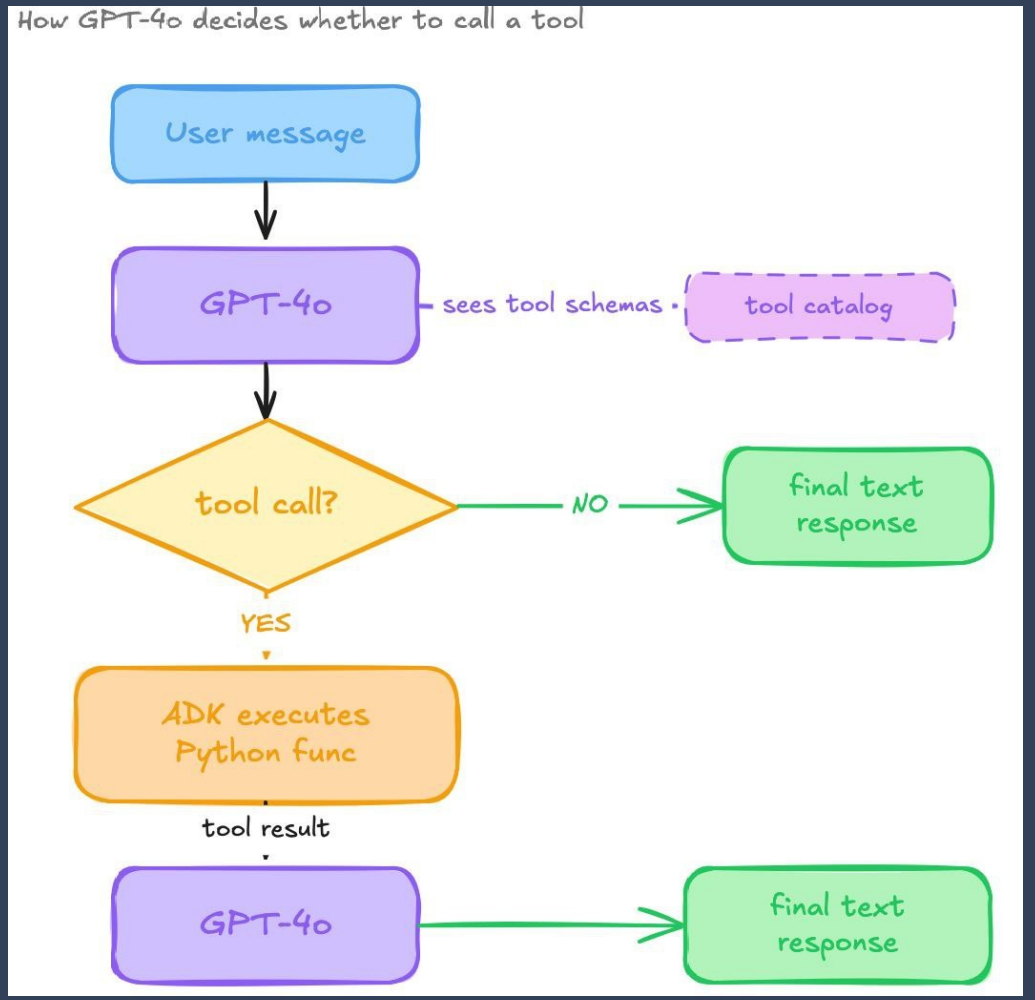
The Simplest ADK Agent

```
root_agent = LlmAgent(
    name="basic_agent",
    model="openai/gpt-4o",
    instruction="Use get_quick_estimate when asked about properties.",
    tools=[get_quick_estimate],    # plain Python function
)
```

KEY CONCEPTS

- LlmAgent = model + instruction + tools
- root_agent = the variable adk web discovers
- Function tools: ADK reads type hints + docstring → JSON schema
- The LLM decides WHETHER to call tools — not scripted

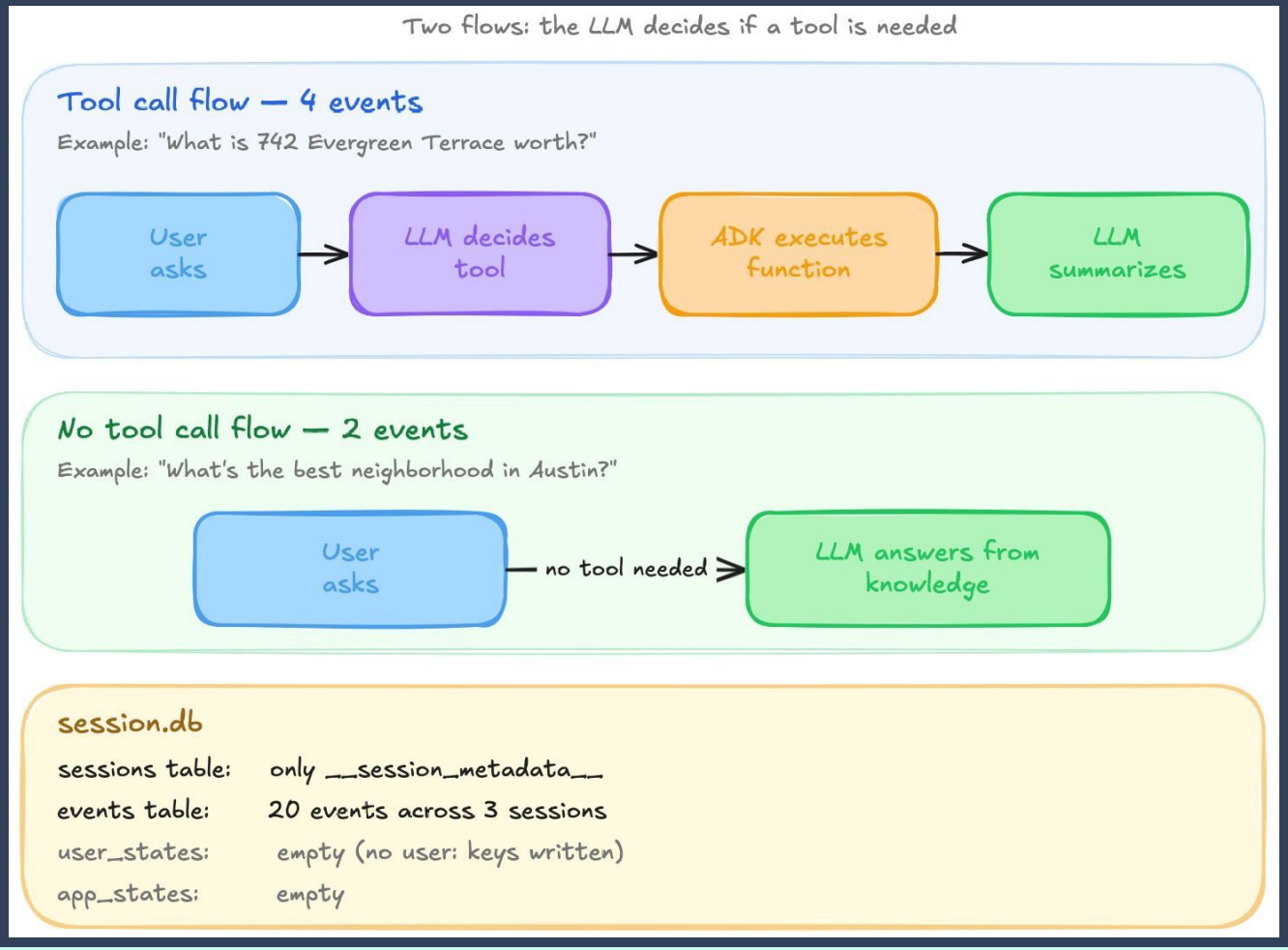
Try this	Expected
"What's 742 Evergreen Terrace worth?"	Calls tool → $462K, high confidence
"What about 999 Unknown Ave?"	Calls tool → $450K, LOW confidence
"What's the best neighborhood in Austin?"	NO tool call — LLM knowledge



{ik} INTERVIEW KICKSTART

Demo 01: Results

What Happened



KEY TAKEAWAY Compare to M1's github_agent_client.py — there you hand-wrote the tool loop (call LLM → check tool_calls → execute → feed back). Here, tools=[fn] and ADK does the rest. ADK replaces ~60 lines of manual tool loop with one declaration.

{ik} INTERVIEW KICKSTART

Demo 02: MCP Tools in ADK — Remote + Authenticated

Auto-discover tools from a protected MCP server

```
root_agent = LlmAgent(
    name="mcp_tools_agent",
    model="openai/gpt-4o",
    tools=[McpToolset(
        connection_params=StreamableHTTPConnectionParams(
            url="http://127.0.0.1:8001/mcp",    # --http server
            headers={"Authorization":
                     f"Bearer {PRICING_MCP_TOKEN}"},
            timeout=30.0,
        ))],
    before_model_callback=_require_tools,
)
```

Layer	Rule	What you see
Authentication	Valid, signed, unexpired JWT	Bad token → 401, ADK loads no tools
Server authorization	Token needs pricing:read	guest token → 403, no tools
Tool authorization	calculate_discount needs pricing:analyze	buyer: market price ✓, discount ✗ "forbidden"

```
# Terminal 1 — protected pricing server
python m1_mcp/pricing_server.py --http
# Pick an identity (writes the token to .env)
python utils/pricing_auth.py --role buyer --write-env
# Terminal 2
adk web m2_adk_multiagents/adk_demos/
```

d01: tools=[get_quick_estimate] — you wrote the function.
d02: tools=[McpToolset(...)] — ADK connects to the server and discovers the tools via MCP. Same function schemas for the LLM.

{ik} | INTERVIEW KICKSTART

Demo 02: Results

Same agent, different token — the server decides (actual runs, gpt-4o)

#	Token	Question	What happened
1	buyer	What's 742 Evergreen Terrace worth?	Calls get_market_price → $462K estimate, comps
2	buyer	Calculate the discount range for a $485K listing (balanced, 18 days, good)	calculate_discount → forbidden: lacks scope 'pricing:analyze'. The model names the missing scope
3	analyst	Same discount question	calculate_discount runs → 2%–5% range, suggested offers
4	invalid	What's 742 Evergreen Terrace worth?	Server 401 → ADK loads no tools → "I can't reach the pricing tools"
5	any valid	What's the crime rate near 742 Evergreen?	No tool call — the agent has no crime tool

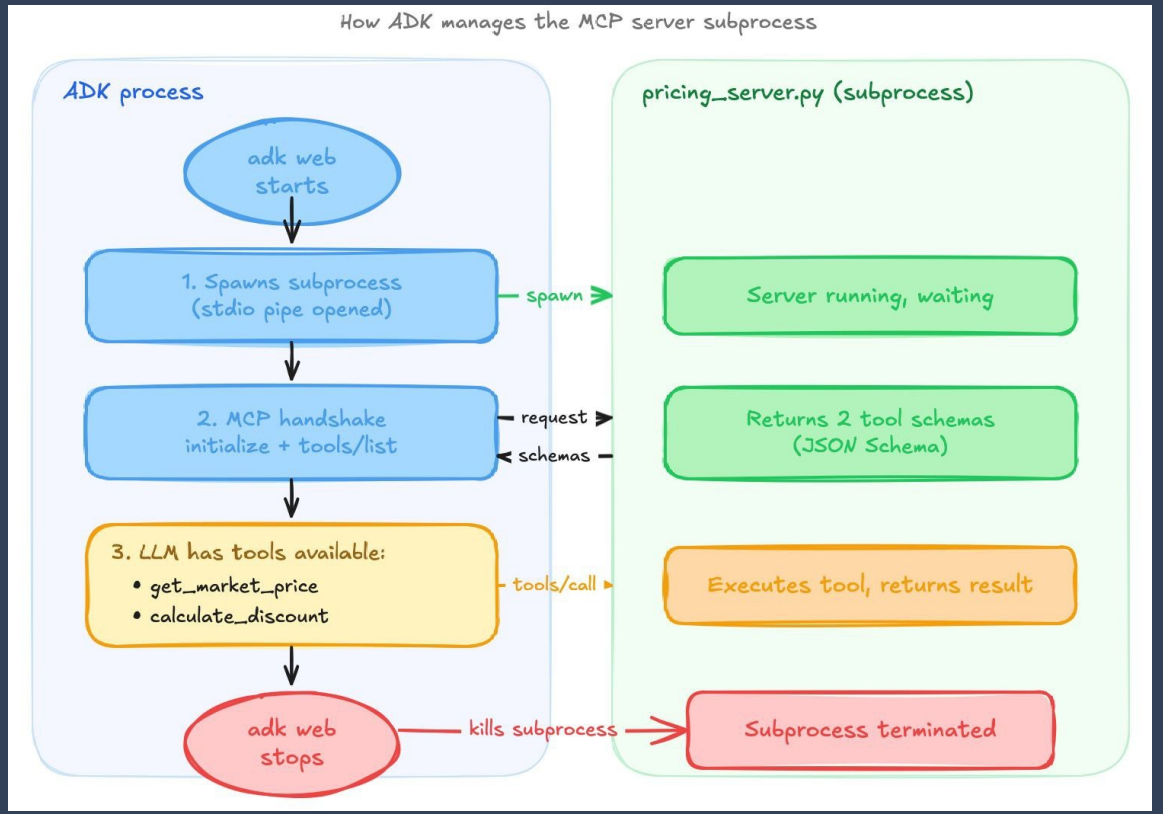
KEY TAKEAWAYS Tools are discovered, never named in agent.py — add an @mcp.tool() and restart. Authorization errors come back as tool results the model can explain. A failed connection means zero tools, so the before_model_callback answers honestly instead of letting the model pretend.

{ik} INTERVIEW KICKSTART

McpToolset: Two Ways to Connect

	StdioConnectionParams	StreamableHTTPConnectionParams
Who starts the server	ADK spawns it as a subprocess	You run it (or it's hosted)
Used by	buyer_agent, seller_agent, negotiation	d02 (pricing_server.py --http)
Auth	None — stdio has no auth layer	Bearer token in headers on every request
Lifecycle	Dies with adk web	Independent process; many clients

The diagram shows the stdio path the buyer and seller use: adk web spawns pricing_server.py, runs the MCP handshake, hands the tools to the LLM, and kills the subprocess on shutdown.



{ik} INTERVIEW KICKSTART

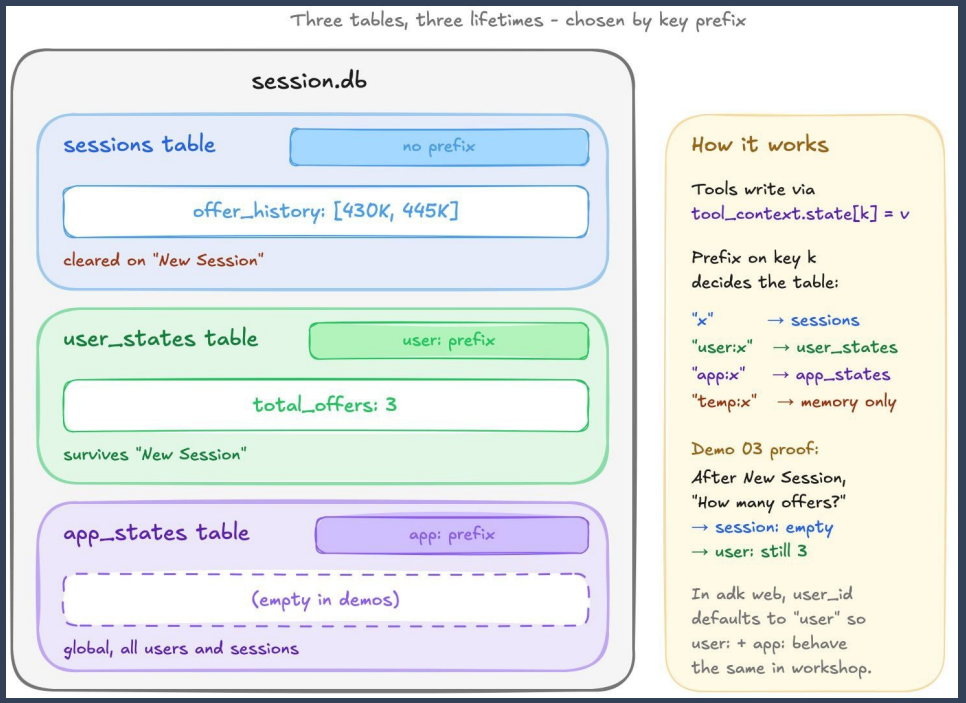
Demo 03: Sessions & State (Concept)

Tools That Remember — the key prefix picks the scope

```python
def record_offer(price: int, tool_context: ToolContext) -> dict:
    state = tool_context.state
    state["offer_history"] = [*state.get("offer_history", []), price]   # session
    state["user:total_offers"] = state.get("user:total_offers", 0) + 1  # this user
    state["app:total_offers"] = state.get("app:total_offers", 0) + 1    # everyone
```

Prefix	Shared with	Table in .adk/session.db
(none)	This conversation only	sessions
user:	This user, across their sessions	user_states
app:	Every user of this agent	app_states
temp:	This invocation only	never saved

Type (same session)	State afterwards
"I offer $430,000"	offer_history [430000]; user/app totals 1
"I offer $445,000"	history [430000, 445000]; totals 2
New Session	no offer_history; user: and app: still 2
"I offer $450,000"	history [450000]; totals 3



{ik} INTERVIEW KICKSTART

Demo 03: Results

Scopes proven by New Session

- **One tool call, three tables.** record_offer writes offer_history, user:total_offers and the app: keys — the prefix, not the tool, decides where each lands
- **New Session proves the scopes.** Before you type anything, the State tab already shows user: and app: keys, while offer_history is gone
- **Prefix stripped in storage.** user:total_offers is stored as total_offers in the user_states row — the table is the scope

Why user: and app: look the same here	After alice's 3 offers	Then bob makes 1 offer
alice's user:total_offers	3	3 (bob has his own row)
bob's user:total_offers	—	1
app:total_offers (everyone)	3	4 (one shared row)

adk web is single-user (user_id = "user"), so user: and app: move together in the UI. A real app passes each person's user_id to runner.run_async(...) — then they diverge.

{ik} | INTERVIEW KICKSTART

Demo 04: SequentialAgent (Concept)

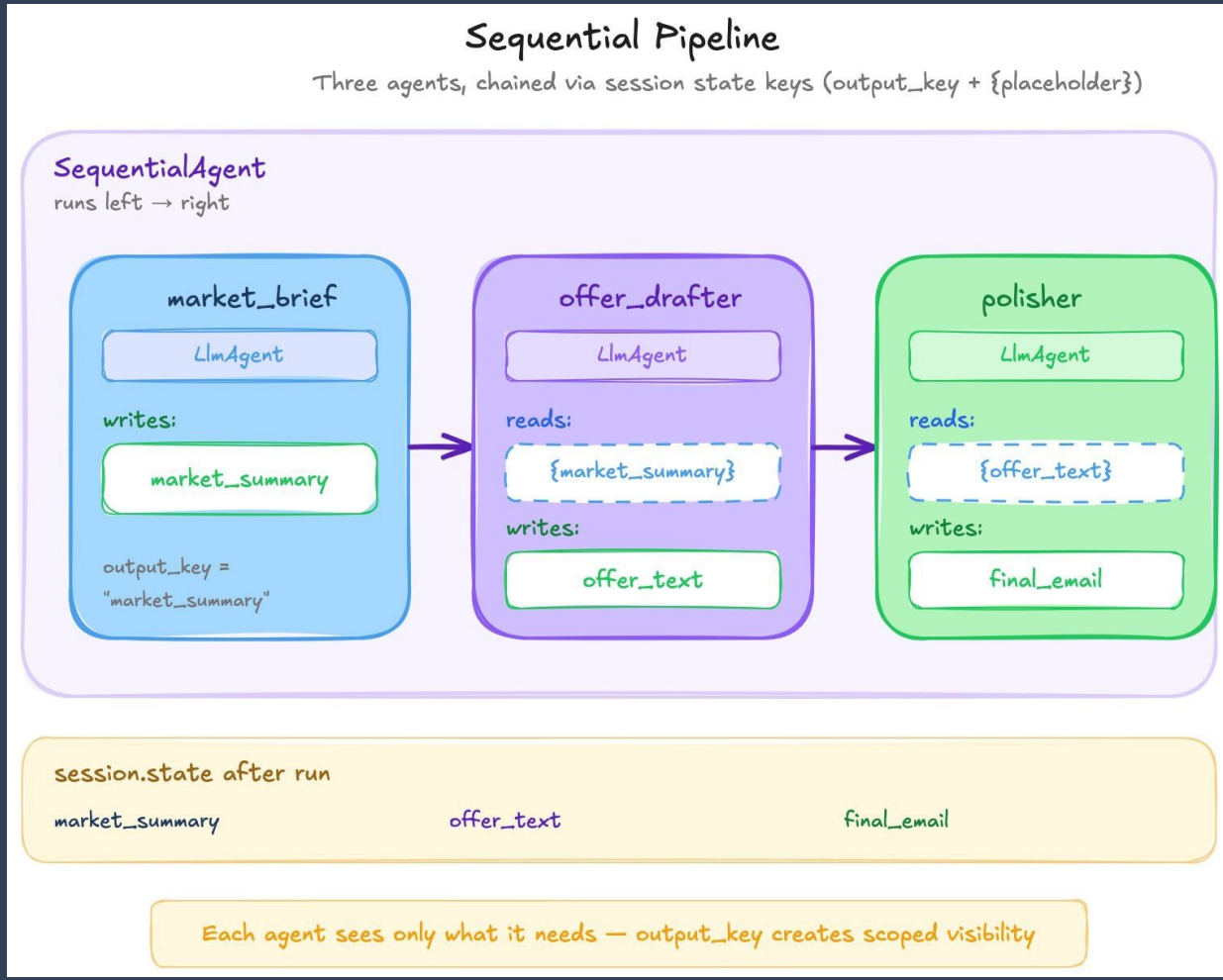
Pipeline: A → B → C

```
market_brief = LlmAgent(output_key="market_summary", ...)
offer_drafter = LlmAgent(output_key="offer_text",
    instruction="Read {market_summary} and draft an offer...")
polisher = LlmAgent(output_key="final_email",
    instruction="Polish {offer_text} into a professional email...")
root_agent = SequentialAgent(
    sub_agents=[market_brief, offer_drafter, polisher],
)
```

DATA FLOW VIA STATE

```
market_brief writes "market_summary"
  → offer_drafter reads {market_summary}, writes "offer_text"
    → polisher reads {offer_text}, writes "final_email"
```

ADK 2.x: SequentialAgent still works but is deprecated in favor of Workflow (see after Demo 06).



{ik} INTERVIEW KICKSTART

Demo 04: Results

Three Agents, One Pipeline

```
Event 1 | user              -> "Research this property"
Event 2 | market_brief      -> no tools, so LLM knowledge only: sometimes
                               "I'm unable to access current data", sometimes
                               invented figures (one run: a "$720,000 median")
Event 3 | offer_drafter     -> "We extend an offer of $460,000..."
Event 4 | message_polisher  -> "Subject: Initial Offer..."
```

- State tab after the run: market_summary, offer_text, final_email — every intermediate output is visible
- Each agent saw only what it needed: offer_drafter read {market_summary}, not the polisher's output

KEY TAKEAWAY State keys create scoped visibility. And an agent with no data tools is not grounded — whether it refuses or invents. That's what MCP tools fix. negotiation/agent.py uses SequentialAgent(buyer, seller).

Demo 05: ParallelAgent (Concept)

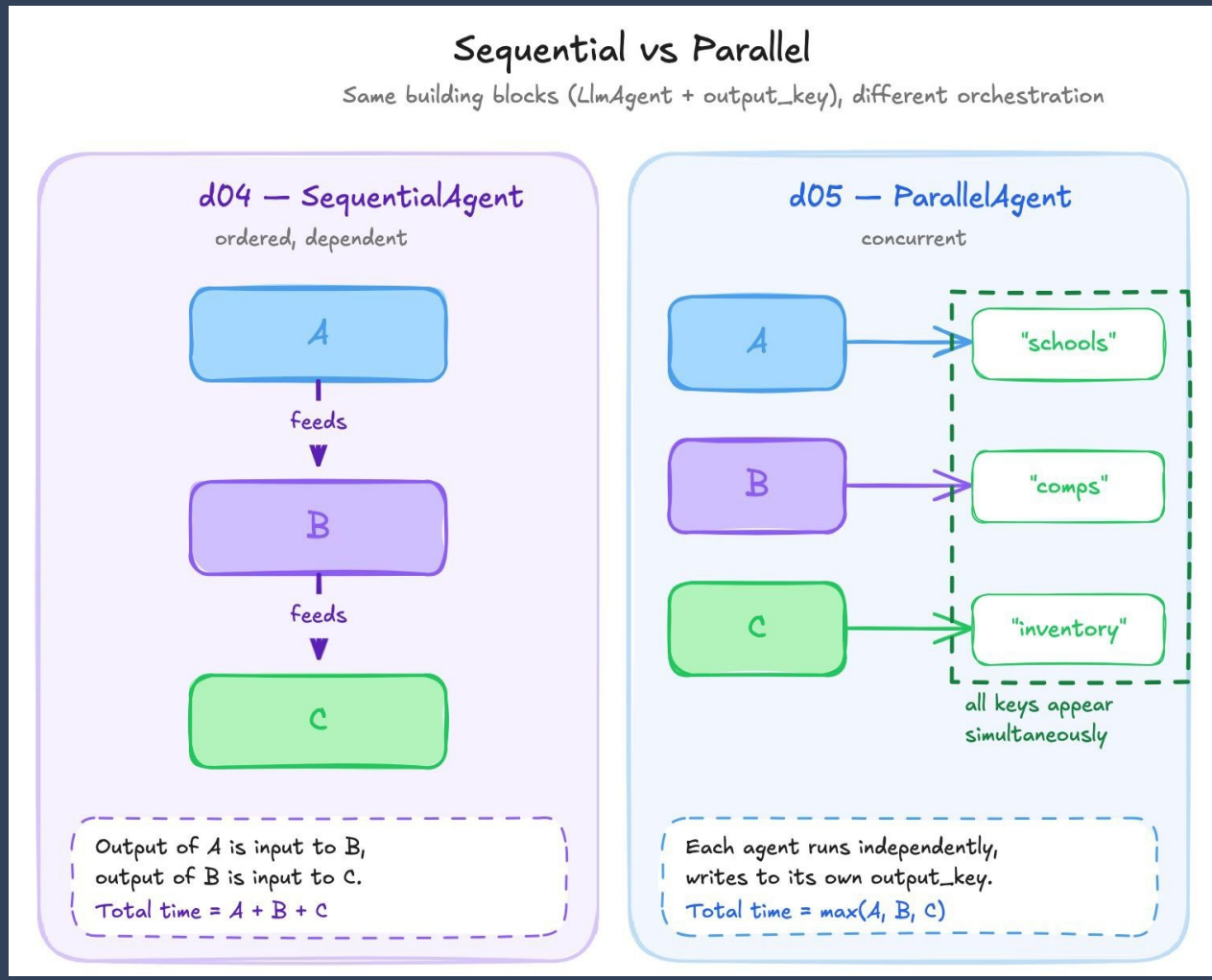
Fan-Out: A, B, C Run Concurrently

```
schools = LlmAgent(output_key="schools", ...)
comps = LlmAgent(output_key="comps", ...)
inventory = LlmAgent(output_key="inventory", ...)

root_agent = ParallelAgent(
    sub_agents=[schools, comps, inventory],  # all at once
)
```

- d04 Sequential: A feeds B feeds C (pipeline)
- d05 Parallel: A, B, C run simultaneously (fan-out)

Rule: each agent MUST write to a different output_key. Two agents writing the same key → last one wins.



Demo 05: Results

Independent Signals Gathered

```
schools:    "Austin ISD has strong academic programs..."
comps:      "Recent sales ..."            # LLM knowledge — numbers vary run to run
inventory:  "Inventory pressure ..."
```

Same 4-event structure as d04 — but concurrent execution. The order the three answers finish varies from run to run.

KEY TAKEAWAY ParallelAgent = fan-out for independent tasks. SequentialAgent = pipeline for dependent tasks. Both use output_key. No {placeholder} reading between parallel agents — they're independent, which is why ParallelAgent works here.

Demo 06: LoopAgent (Concept)

Iterate Until Done

```
def stop_when_in_range(callback_context: CallbackContext):
    price = int("".join(c for c in callback_context.state.get("proposal", "") if c.isdigit()))
    if 450_000 <= price <= 470_000:
        callback_context.state["loop_exit_reason"] = f"{price:,} is in range"  # required!
        callback_context.actions.escalate = True                               # breaks the loop

haggler = LlmAgent(name="haggler", output_key="proposal",
                   after_agent_callback=stop_when_in_range, ...)
root_agent = LoopAgent(sub_agents=[haggler], max_iterations=5)   # bounded
```

TWO STOP CONDITIONS

- callback_context.actions.escalate = True — early exit (price in range)
- max_iterations=5 — hard cap (same guarantee as an FSM's max_turns)

ADK gotcha — a bare escalate is silently dropped. A callback only becomes an event if it returns content or changes state. Set escalate alone and the loop keeps running (all 5 iterations). Always pair it with a state write. Verified on ADK 1.39 and 2.11.

{ik} | INTERVIEW KICKSTART

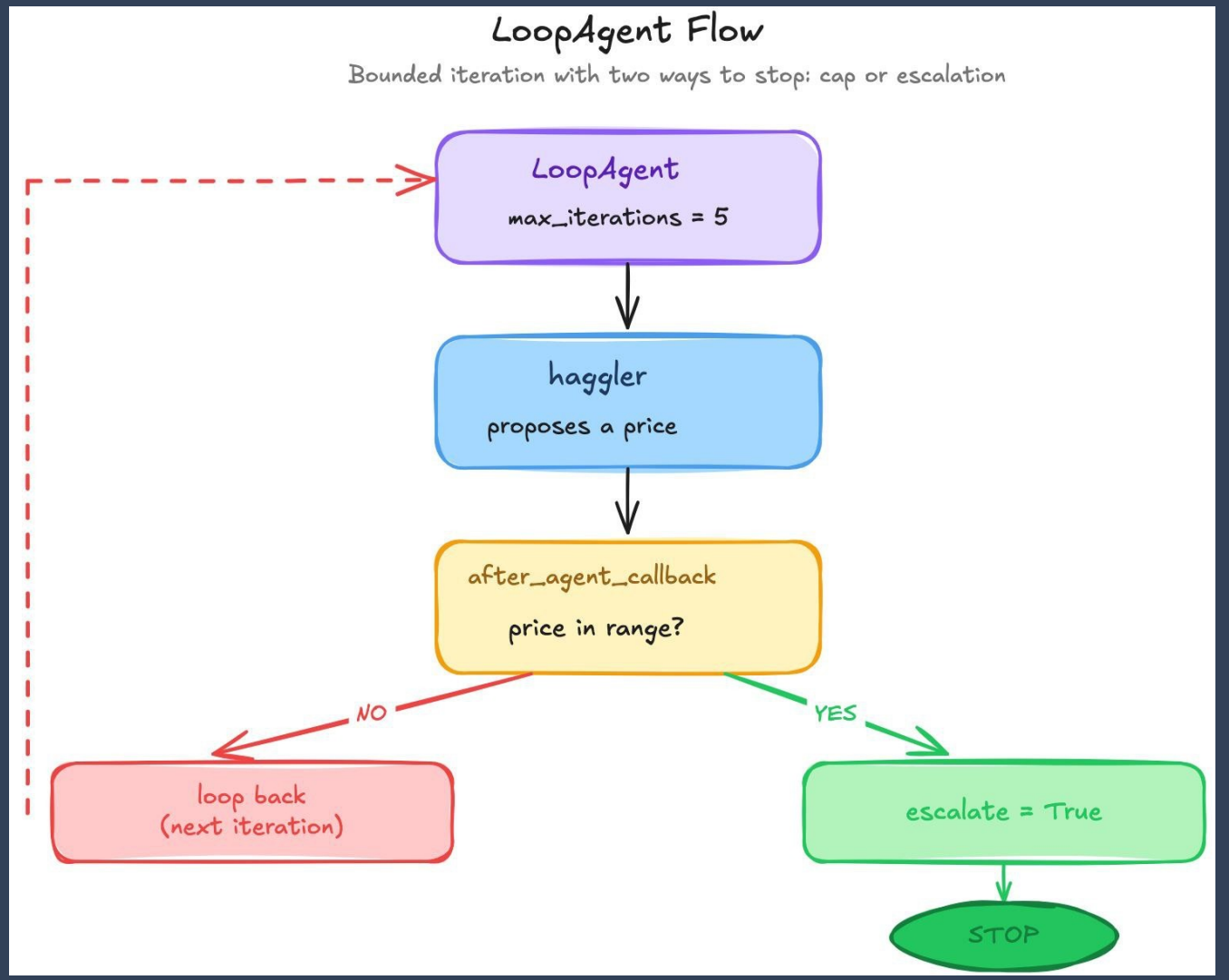
Demo 06: Results

Bounded Termination

```
Event 1 | user:    "start"
Event 2 | haggler: "465000"   state_delta {proposal}
Event 3 | haggler: (no text)  state_delta {loop_exit_reason}
                              escalate=True -> LoopAgent STOPS
```

- Event 3 exists only because the callback wrote state — that event carries the escalation
- output_key in a loop overwrites the same key each iteration; only the last value survives
- Worst case: 5 iterations, never more

negotiation/agent.py uses LoopAgent(SequentialAgent(buyer, seller), max_iterations=5). Each iteration is one round.



{ik} INTERVIEW KICKSTART

ADK 2.x: Workflow Replaces Sequential / Parallel / Loop

In ADK 2.x, SequentialAgent, ParallelAgent and LoopAgent still work (this workshop uses them) but print a DeprecationWarning: they are deprecated in favor of Workflow — a graph of nodes and edges in google.adk.workflow.

```python
from google.adk.workflow import Workflow, JoinNode

# nodes: plain functions, LlmAgents or tools
def offer(ctx, node_input):
    n = ctx.state.get("round", 0) + 1
    ctx.state["round"] = n
    ctx.route = "done" if n >= 3 else "again"
    return f"offer {n}"

join = JoinNode(name="join")     # waits for every branch
wf = Workflow(name="demo", edges=[
    ("START", research, (comps, schools), join, offer),
    (offer, {"again": offer, "done": finish}),  # routed
])
```

Old	Workflow equivalent
SequentialAgent(a, b, c)	a chain: ("START", a, b, c)
ParallelAgent(a, b)	a tuple (a, b) fans out; JoinNode waits for both
LoopAgent + escalate	a cycle with a routed edge; a node sets ctx.route
max_iterations	your own counter in the routing node

Rules: every cycle needs at least one routed edge (unconditional cycles are rejected); a Workflow can't yet be an LlmAgent's sub-agent. Verified: this code (with stub research/comps/schools/finish functions) ran on ADK 2.11.

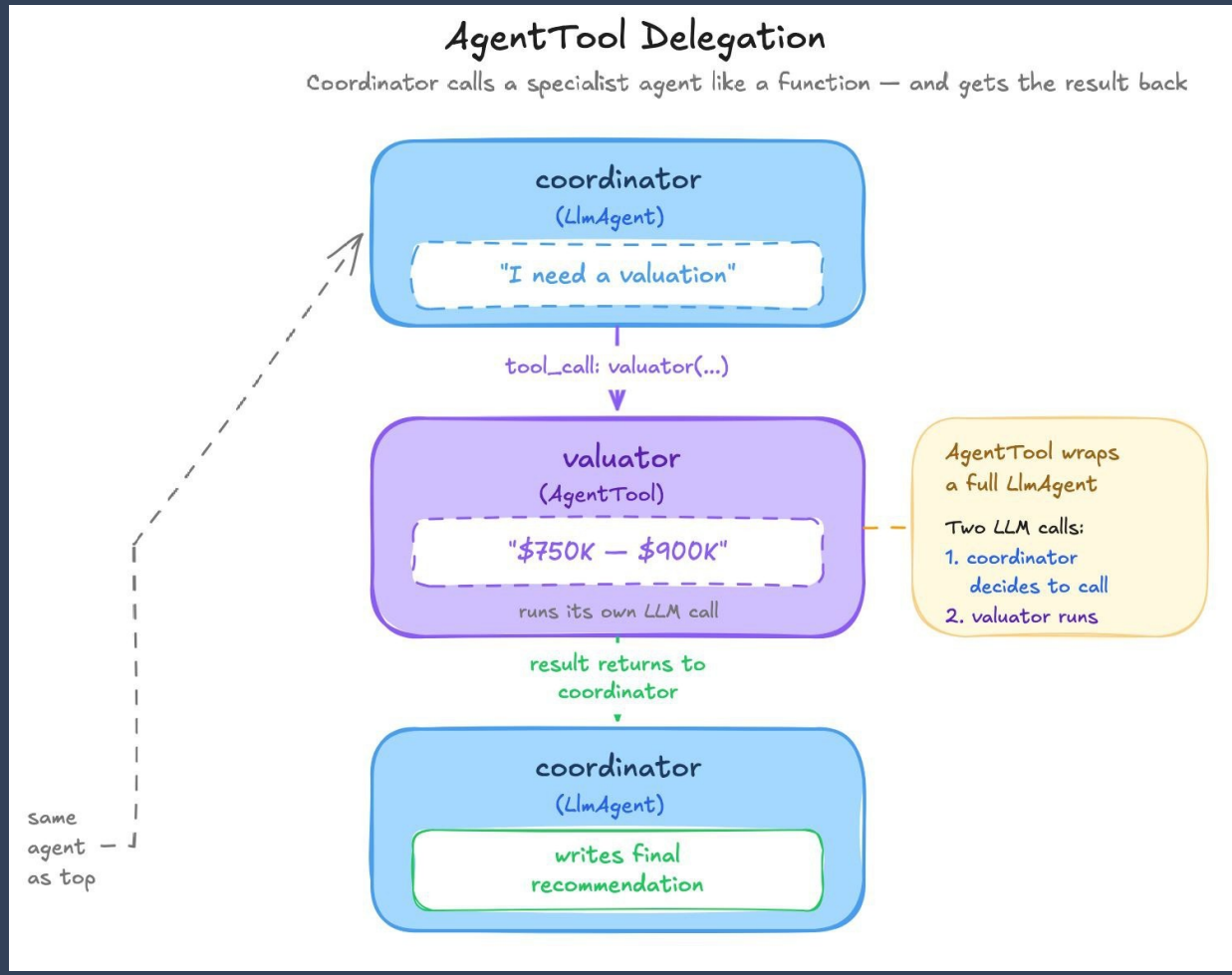
Demo 07: Agent-as-Tool (Concept)

Wrap an Agent as a Callable Tool

```
valuator = LlmAgent(
    name="valuator",
    description="Estimates fair market value of Austin properties.",
)
root_agent = LlmAgent(
    name="coordinator",
    tools=[AgentTool(agent=valuator)],  # agent as a tool
)
```

Pattern	How it works	Control
AgentTool	Call agent as a function → result returns	Caller keeps control
sub_agents	Transfer to agent → it takes over	Full delegation
SequentialAgent	Fixed order, no runtime choice	No decision

Try	Expected
"What should I offer on 742 Evergreen?"	Calls valuator → estimate → recommendation
"What's the weather?"	No valuator call — not relevant



{ik} INTERVIEW KICKSTART

Demo 07: Results

Delegation in Action

```
Event 1 | user        -> "What should I offer?"
Event 2 | coordinator -> tool_call: valuator({"request": "742 Evergreen..."})
Event 3 | coordinator -> tool_result: "$750K-$900K, location is biggest factor"
Event 4 | coordinator -> "I recommend offering within $750K-$900K range..."
          (the range varies run to run - another run said $1.2M-$1.5M)
```

NOTICE The valuator has no data tools, so it estimates from LLM knowledge — and the number isn't even stable between runs. d02's MCP-backed get_market_price returned $462K from data. This is why MCP matters.

KEY TAKEAWAY AgentTool = explicit delegation: the coordinator decides WHEN to call the specialist, and reads its description like a function docstring. Two LLM calls per delegation — more expensive, cleaner separation.

Demo 08: Callbacks (Concept)

Policy Without Prompts — five hooks, each doing something visible

Callback	When it fires	Policy in this demo	Return to override
before_agent_callback	Once per turn	Count turns	Content → skip the agent
before_model_callback	Before every LLM call	Redact SSNs from the request	LlmResponse → skip the LLM
before_tool_callback	Before every tool call	Block get_internal_admin	dict → skip the tool
after_tool_callback	After every tool returns	Strip seller_floor_usd from results	dict → replace the result
after_model_callback	After every LLM response	Remove phone numbers from the reply	LlmResponse → replace it

Try (same session)	What happens
"My SSN is 123-45-6789. What's 742 Evergreen Terrace worth?"	SSN redacted; tool allowed; floor stripped; phone → [phone removed]
"What's the seller's minimum acceptable price?"	Model calls get_internal_admin → before_tool BLOCKS it
"Who is the listing agent and how do I contact them?"	Name shared, phone number removed

The instruction mentions none of these rules. Callbacks are deterministic; instructions are suggestive.

Demo 08: Results

Security Audit Trail (actual run, gpt-4o)

```
Turn 1: "My SSN is 123-45-6789. What's 742 Evergreen Terrace worth?"
  [before_agent] turn 1 started
  [before_model] redacted SSN in 1 message(s)     -> LLM sees [REDACTED-SSN]
  [before_tool]  allowed get_quick_estimate
  [after_tool]   stripped ['seller_floor_usd']    -> LLM never sees the floor
  [before_model] redacted SSN in 1 message(s)     -> history still has it: scrubbed again
  [after_model]  removed phone number(s) from the reply
Turn 2: "What's the seller's minimum acceptable price for 742 Evergreen Terrace?"
  LLM -> get_internal_admin(...)                  <- the model DID try
  [before_tool]  BLOCKED get_internal_admin       -> "I'm unable to access..."
```

KEY TAKEAWAY Callbacks = deterministic policy. before_model runs on EVERY LLM call; redaction edits only the model's copy (the stored event keeps the SSN). This is the pattern buyer_agent and seller_agent use for tool allowlists. In production, swap print() for structured logging.

{ik} | INTERVIEW KICKSTART

Demo 09: Event Stream (Concept)

See What the Runner Produces

```
def lookup_comps(address: str, tool_context: ToolContext) -> dict:
    tool_context.state["last_comp_lookup"] = address
    return {"comps": [...], "avg_comp_price": 465_000}

def estimate_offer(comp_avg: int, discount_pct: int, tool_context: ToolContext) -> dict:
    tool_context.state["latest_offer"] = int(comp_avg * (1 - discount_pct / 100))
    tool_context.state["offer_count"] = tool_context.state.get("offer_count", 0) + 1
```

- Web UI: State tab (updates live) + left panel event inspector (state deltas)
- Terminal: python m2_adk_multiagents/adk_demos/d09_event_stream/run_event_stream.py prints every Event
- Try: "What should I offer on 742 Evergreen Terrace?" → check the State tab

This demo is about seeing state writes and event structure, not the chat response.

{ik} | INTERVIEW KICKSTART

Demo 09: Results

The Event Stream, Field by Field (actual run, gpt-4o)

```
[Event 1] function_call      lookup_comps({"address": "742 Evergreen Terrace, ..."})
[Event 2] function_response  lookup_comps -> {"avg_comp_price": 465000, ...}
          state_delta        {"last_comp_lookup": "742 Evergreen Terrace, ..."}
[Event 3] function_call      estimate_offer({"comp_avg": 465000, "discount_pct": 5})
[Event 4] function_response  estimate_offer -> {"offer_price": 441750}
          state_delta        {"latest_offer": 441750, "offer_count": 1}
[Event 5] final_response=True  "... I recommend making an offer of $441,750."
```

- The model chained the tools: estimate_offer got the real comp average ($465K) → $441,750
- State deltas ride on the tool-RESULT events; final_response=True appears once per turn
- run_async yields 5 events; the session stores 6 (your message too)

Parallel tool pitfall: if a model emits both calls at once, it must guess comp_avg before seeing the lookup. When tool B depends on tool A, enforce the order — a SequentialAgent (or Workflow chain).

No Question Is Too Small Every Question Deserves an Answer!!

Ask about anything:

- Concepts from today's modules
- How this applies to your work
- Implementation details
- Clarifications on what we covered

Quick Request

- Please do not mind if a question gets skipped accidentally
- Repost politely in chat if needed
- We will try our best to cover as many questions as possible

Remember: If you're wondering about it, others probably are too.



Time for a Break



A 5-minute break is starting now. Please return when the timer ends to continue the presentation

{ik} | INTERVIEW KICKSTART

Live Zoom Poll



We are launching a poll on Zoom. Please take 2 minutes to fill out the form.

{ik} | INTERVIEW KICKSTART

What is A2A?

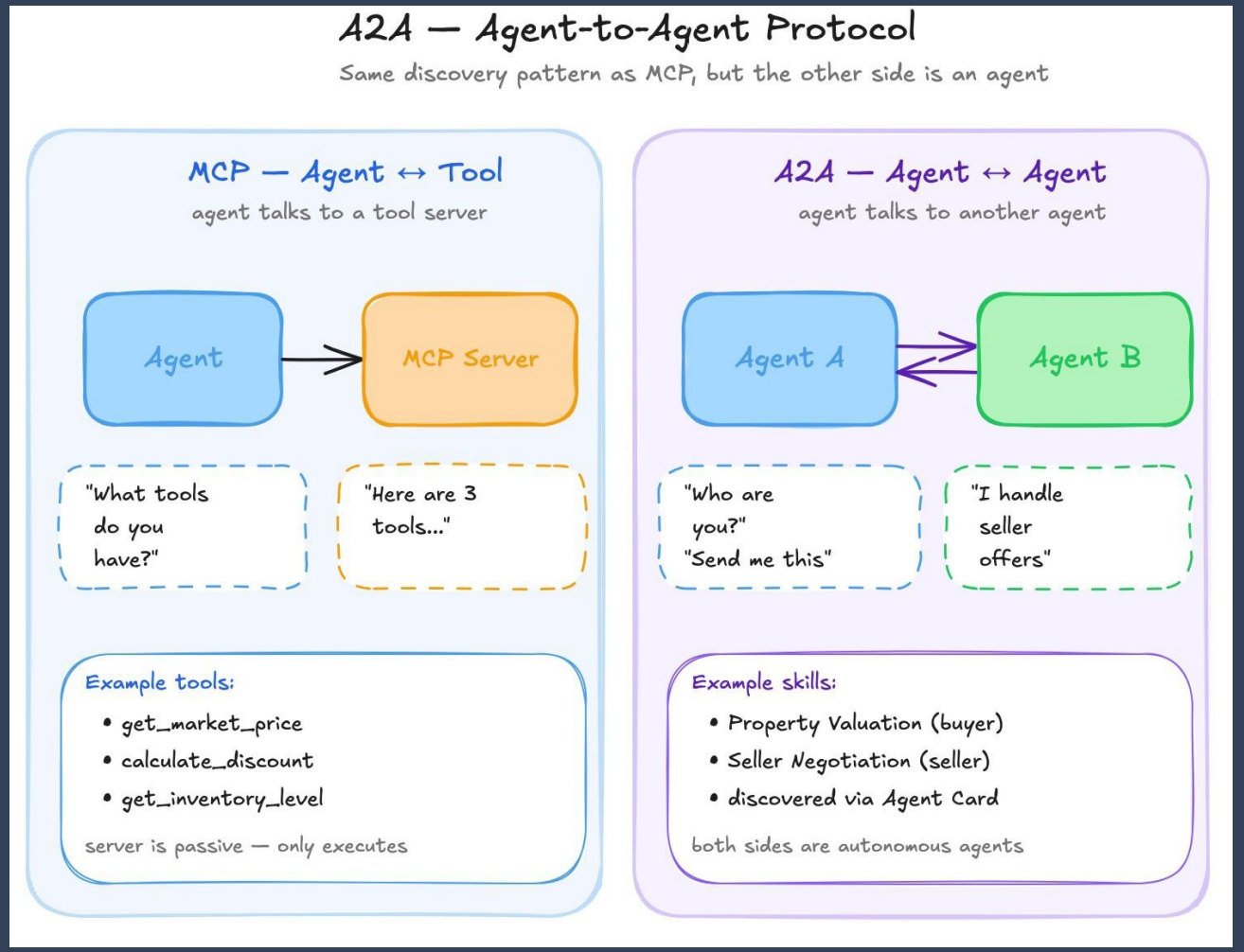
Agent-to-Agent Protocol

The problem: so far all our agents run in the SAME process — buyer and seller are Python objects in one adk web instance. What if they're on different machines, teams, languages?

A2A solves this: an open protocol for agents to discover each other and communicate over HTTP.

THREE OPERATIONS (SAME PATTERN AS MCP)

- **Discovery:** GET /.well-known/agent-card.json — "Who are you and what can you do?"
- **Schema:** the Agent Card — name, description, skills, capabilities
- **Invocation:** POST to the card's url (JSON-RPC message/send) — "Handle this task"



{ik} INTERVIEW KICKSTART

Why A2A Matters

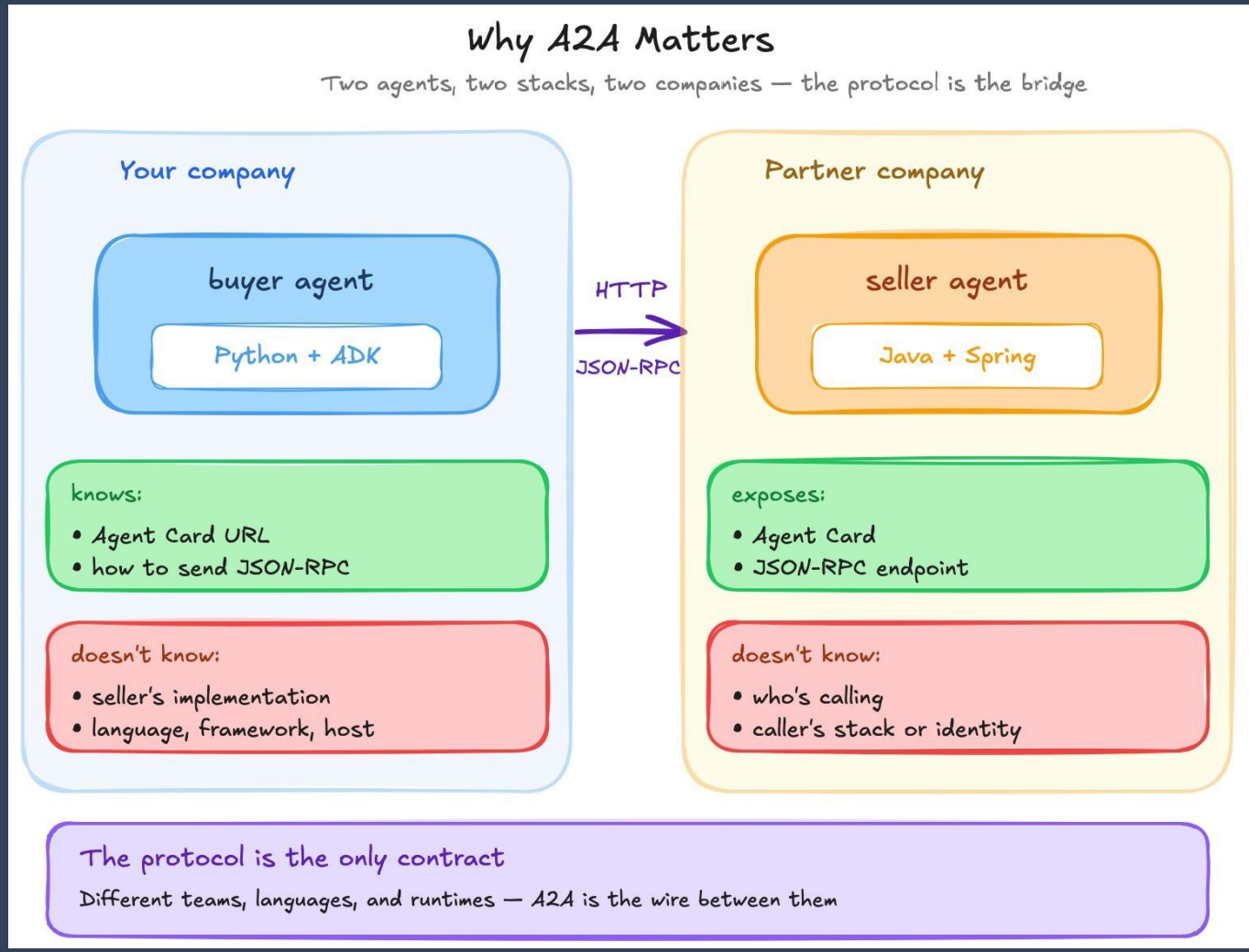
From In-Process to Network Services

WITHOUT A2A (d01–d09)

- Buyer and seller are Python objects in the same process
- They communicate via shared session state (output_key, {placeholder})
- Tightly coupled — you must import both to run either

WITH A2A

- Buyer runs on machine A, seller on machine B
- They communicate via HTTP JSON-RPC
- Neither knows how the other is built — only its Agent Card
- Could be different languages, teams, companies



{ik} INTERVIEW KICKSTART

Agent Card

Agent Card = the agent's business card

Every A2A agent publishes a JSON document at /.well-known/agent-card.json. With adk web --a2a, it's the agent.json next to agent.py:

```
{
  "name": "seller_agent",
  "description": "Real estate seller agent for 742 Evergreen Terrace...",
  "url": "http://localhost:8000/a2a/seller_agent",
  "preferredTransport": "JSONRPC",
  "protocolVersion": "0.3.0",
  "capabilities": {"streaming": true, "pushNotifications": false},
  "skills": [{
    "id": "negotiation_response",
    "name": "Real Estate Seller Negotiation",
    "tags": ["real_estate", "seller", "negotiation", "mcp"]
  }]
}
```

WHAT A CLIENT LEARNS FROM THE CARD

- What this agent does (description + skills)
- Where to send messages (url) and how (preferredTransport)
- Which optional features it supports (capabilities)
- No documentation needed — self-describing, like MCP tool schemas

{ik} | INTERVIEW KICKSTART

Agent Card Capabilities: All Four, and What They Turn On

Capability	Meaning (spec v0.3.0)	What it turns on	Our cards
`streaming`	Supports Server-Sent Events	message/stream, tasks/resubscribe	seller: true · buyer, negotiation: false
`pushNotifications`	Can POST task updates to a client webhook	tasks/pushNotificationConfig/ set·get·list·delete	false everywhere (not demoed)
`stateTransitionHistory`	Provides a history of a task's state transitions	Informational	not set
`extensions`	Protocol extensions (uri, description, required, params)	Extension-specific	not set

- **Not a capability:** message/send, tasks/get and tasks/cancel always exist — so polling works with any agent
- **Enforced by the a2a-sdk server:** only message/stream (needs streaming) and pushNotificationConfig/set (needs pushNotifications). Both refusals come back as JSON-RPC -32603
- **The SDK Client reads the card too:** with ClientConfig(streaming=True) it streams only if card.capabilities.streaming is true, else falls back to message/send

Other card fields advertise features too: securitySchemes / security (how to authenticate), supportsAuthenticatedExtendedCard, additionalInterfaces, signatures.

{ik} | INTERVIEW KICKSTART

How Clients Find Agent Cards

Three discovery strategies (A2A docs)

Well-known URI	Curated registry	Direct configuration
GET https://host/.well-known/agent-card.json — the agent's own domain advertises it	Agents publish cards to a catalog; clients search by skill, tag or keyword (e.g. Google Cloud Agent Registry, AGNTCY Agent Directory)	The client is told the URL — what our demos do with --seller-url

The A2A spec doesn't define a registry API — every registry has its own. Whatever the source, the client ends up with the same thing: an Agent Card, and it reads url, skills, capabilities and securitySchemes to decide how to call the agent.

That's why rich, accurate skill descriptions and tags in agent.json matter: they are what gets searched.

{ik} INTERVIEW KICKSTART

Demo 10: A2A Wire Format & the Task (Concept)

Agent Card → the real HTTP bytes of message/send → the Task, taken apart

```
adk web --a2a m2_adk_multiagents/negotiation_agents/          # terminal 1
python m2_adk_multiagents/a2a_demos/a2a_10_wire_lifecycle.py   # terminal 2
```

```
card = await A2ACardResolver(httpx_client=http, base_url=seller_url).get_agent_card()
client = ClientFactory(ClientConfig(httpx_client=http, streaming=False)).create(card)

message = Message(message_id="msg_7af7c873", role=Role.user, parts=[
    TextPart(text="Opening offer at $440k, 30-day close, pre-approved."),
    DataPart(data={"round": 1, "message_type": "OFFER", "price": 440000, ...}),
])
async for event in client.send_message(message):    # non-streaming: one (Task, None)
    task = event[0]
```

- ClientFactory + ClientConfig → Client is the a2a-sdk's current client (A2AClient is deprecated)
- The request goes to card.url over card.preferredTransport — the card drives the client
- httpx event hooks print the exact bytes sent and received — nothing re-built

```
-> POST http://localhost:8000/a2a/seller_agent
{"jsonrpc": "2.0", "id": "b23808c5-...", "method": "message/send",
 "params": {"configuration": {"blocking": true}, "message": {"parts": [ ... ]}}}
```

{ik} INTERVIEW KICKSTART

Demo 10: Results

The Task, taken apart (actual run, gpt-4o)

```
id          162a296b-5515-4458-9601-4eb8b5af4bb5    <- this unit of work (tasks/get uses it)
contextId   0918bf29-218b-4011-86b2-e21b2dd517c4    <- the conversation it belongs to
status      completed
history     7 message(s):
  [0] user  text + data (the offer)
  [1] user  text + data (same message again - an ADK quirk)
  [2] agent data (tool_call get_market_price, get_inventory_level, get_minimum_acceptable_price)
  [3] agent data (tool_result x3)
  [4] agent data (tool_call submit_decision)    [5] agent data (tool_result submit_decision)
  [6] agent text "SELLER: We COUNTER at $477,000 ..."
artifacts   1: artifactId=bee4354b-...  text "SELLER: We COUNTER at $477,000 ..."
```

- **The Task is the response:** id, contextId, status, history, artifacts
- **History is the audit trail** — including the seller's MCP tool calls as DataParts
- **The artifact is the deliverable** — read the answer from it

Only the final state is visible — blocking: true answered once the task was completed. Demo 13 shows submitted → working → completed live.

{ik} INTERVIEW KICKSTART

The Task: Lifecycle and States

State	Meaning (spec v0.3.0)	Kind
submitted	Submitted and awaiting execution	active
working	The agent is working on it	active
input-required	Paused, waiting for input from the user	interrupted
auth-required	Needs authentication to proceed	interrupted
completed	Finished successfully	terminal
failed	Failed due to an error during execution	terminal
canceled	Canceled by the user	terminal
rejected	Rejected by the agent and not started	terminal

- **Usual path:** submitted → working → completed (or failed)
- **Terminal means terminal:** a follow-up message in the same contextId creates a NEW task (demo 11)
- **Cancel works** with ADK 2.x's executor: a running task → canceled; cancelling it again is error -32002
- **"completed" ≠ success:** the agent responded — check the content too (demo 14)

JSON-RPC methods: message/send · message/stream · tasks/get · tasks/cancel · tasks/resubscribe · tasks/pushNotificationConfig/ · agent/getAuthenticatedExtendedCard*

Demo 11: Context Threading (Concept)

A task is one job; a context is the conversation

```
round 1  message/send (no contextId)   -> task A, server assigns context X
round 2  message/send (contextId X)    -> task B, context X  (seller sees round 1)
round 3  message/send (contextId X)    -> task C, context X
tasks/get(A)                           -> round 1's task, unchanged
round 4  message/send (no contextId)   -> task D, NEW context Y  (no memory)
```

Round	Offer	What the seller does
1	$432K	Researches (3 MCP tools) → below the $445K floor → COUNTER
2	$440K	Remembers round 1 → only submit_decision → COUNTER
3	$446K	Above the floor → ACCEPT (submit_decision enforces it in code)
4 (no contextId)	$440K	No memory → calls all three MCP tools again → COUNTER

contextId = A2A's session_id. Without it every POST is a brand-new conversation. Round 2 vs round 4 (same $440K) is the proof.

Demo 11: Results

Actual run (gpt-4o, abridged — prices vary run to run)

```
round 1 - $432,000   taskId 3fc7f67a...  contextId 06264516...
          tools: get_market_price, get_inventory_level, get_minimum_acceptable_price, submit_decision
          seller: COUNTER at $477,000
round 2 - $440,000   taskId d6a7db58...  contextId 06264516...   tools: submit_decision
round 3 - $446,000   taskId 5c78919f...  contextId 06264516...   tools: submit_decision  -> ACCEPT
tasks/get(round 1)   taskId 3fc7f67a...  state completed, same artifact
round 4 - $440,000   taskId b7befb6e...  contextId 1de09dfa...   (new conversation)
          tools: get_market_price, get_inventory_level, get_minimum_acceptable_price, submit_decision
```

- **3 rounds → 3 taskIds, 1 contextId.** Each message/send is its own Task; the context links them
- **Round 2 makes no MCP calls; round 4 calls all three** — contextId is the ONLY difference
- **The floor saved in session state in round 1** is still there in round 3, because the context is the same

Demo 12: Parts & Artifacts (Concept)

Multi-part messages, and where the answer lives in a Task

```
parts = [
    TextPart(text="Final-and-best offer at $445k for 742 Evergreen Terrace."),   # for the LLM
    DataPart(data={"hint": "machine-readable copy", "offer": {"price": 445000, ...}}),  # for code
]
```

Part type	Carries	Who reads it
TextPart	Plain text	The LLM
DataPart	Structured JSON	Code / downstream systems
FilePart	A file — inline bytes or a URI, with a MIME type	Depends on the agent (demo 14 sends one it can't read)

Where in the Task	What's there
history	The conversation — incl. MCP tool calls & results as DataParts
status.message	Latest status message (empty when completed; the reason when failed)
artifacts	The deliverable, each with an artifactId — ADK puts the final reply here

Messages = the email thread. Artifacts = the attached report. Read the answer from the artifact; read the audit trail from history.

{ik} | INTERVIEW KICKSTART

Demo 12: Results

Acceptance at $445K (actual run, gpt-4o)

```
=== 1. history (7 messages) ===
  (0) user   TextPart + DataPart (the offer)        (1) the same message again
  (2) agent  DataPart x3 - tool CALLS  get_market_price, get_inventory_level, get_minimum_acceptable_price
  (3) agent  DataPart x3 - tool RESULTS
  (4) agent  DataPart - submit_decision {"action": "ACCEPT", "price": 445000, "buyer_offer": 445000}
  (5) agent  DataPart - {"recorded": "ACCEPT", "price": 445000}
  (6) agent  TextPart - "SELLER: I ACCEPT your offer of $445,000 ..."
=== 2. status.message ===   (none - the final 'completed' update carries no message)
=== 3. artifacts (1) ===     artifactId=79ee2ece-...  TextPart "SELLER: I ACCEPT your offer of $445,000 ..."
```

$445K = exact floor → guaranteed acceptance. submit_decision records any offer at or above the floor as ACCEPT. Tool calls appear as DataParts in history — code can see exactly which tools ran, and what was decided, without parsing text.

{ik} | INTERVIEW KICKSTART

Buyer vs Seller: Information Asymmetry

The design point — and the two controls that enforce it

Data	Buyer	Seller
Market price ($462K)	Yes	Yes
Discount analysis	Yes	Yes
Inventory levels	No	Yes
Floor price ($445K)	**No**	**Yes**

Seller-only tools: get_inventory_level, get_minimum_acceptable_price, submit_decision. The buyer has get_market_price, calculate_discount, submit_offer.

1. What an agent can CALL

before_tool_callback allowlist (d08 pattern). The buyer's allowlist has no get_minimum_acceptable_price — a call is blocked deterministically.

2. What an agent can READ

include_contents="none" on the buyer. In a shared session ADK shows each agent the others' tool results — without this, the buyer reads the seller's floor.

{ik} | INTERVIEW KICKSTART

The Context Leak: An Allowlist Isn't Enough

Buyer and seller share one session in the negotiation orchestrator. By default ADK relays the other agents' turns to each agent as context — tool calls AND tool results:

```
[seller] called tool `get_minimum_acceptable_price` with parameters: {'property_id': '742-evergreen-austin-78701'}
[seller] `get_minimum_acceptable_price` tool returned result: {... "minimum_acceptable_price": 445000 ...}
```

Buyer setting	Buyer LLM calls that contained the seller's floor result	Outcome
include_contents="default"	every call from round 2 on	deal at exactly $445,000 — the floor
include_contents="none"	0 of 9 (3 runs)	offers 425K → 437K → 448K; deals $446K–$453K

```
buyer = LlmAgent(name="buyer", include_contents="none",   # only its current turn
    instruction="... The seller's last response was:\n{seller_response}\n"
                "Your previous offer (empty in round 1):\n{buyer_offer?}", ...)
```

Lesson: information asymmetry needs two controls — what an agent can CALL (allowlists) and what it can READ (its context). In a shared session, the second one is open unless you close it.

{ik} | INTERVIEW KICKSTART

Buyer & Seller: Results

Each agent on its own (actual runs, gpt-4o)

```
BUYER  "What's 742 Evergreen Terrace worth?"
       -> get_market_price -> $462K estimated, $485K list
       "I want to make an offer. What do you recommend?"
       -> calculate_discount(485000, balanced, 18 days, good) -> "~$425,000 (12% below asking)"

SELLER "Buyer offers $440,000 with 30-day close"
       -> get_market_price + get_inventory_level + get_minimum_acceptable_price  (in parallel)
       -> submit_decision(COUNTER, 477000)            "SELLER: We must COUNTER at $477,000..."
       "Buyer increases to $446,000"
       -> submit_decision(COUNTER, 470000)            <- the model tried to counter
       <- {recorded: ACCEPT, price: 446000, note: "...your reply MUST accept it"}
       "SELLER: We ACCEPT the offer of $446,000..."
```

A rule that must hold lives in code. The instruction says accept at or above the floor; the model still countered $446K. submit_decision enforces the floor and records ACCEPT. The seller's three lookups are independent, so calling them in parallel is fine (unlike d09).

{ik} INTERVIEW KICKSTART

Negotiation Orchestrator: Composition

Every demo in one file — negotiation/agent.py

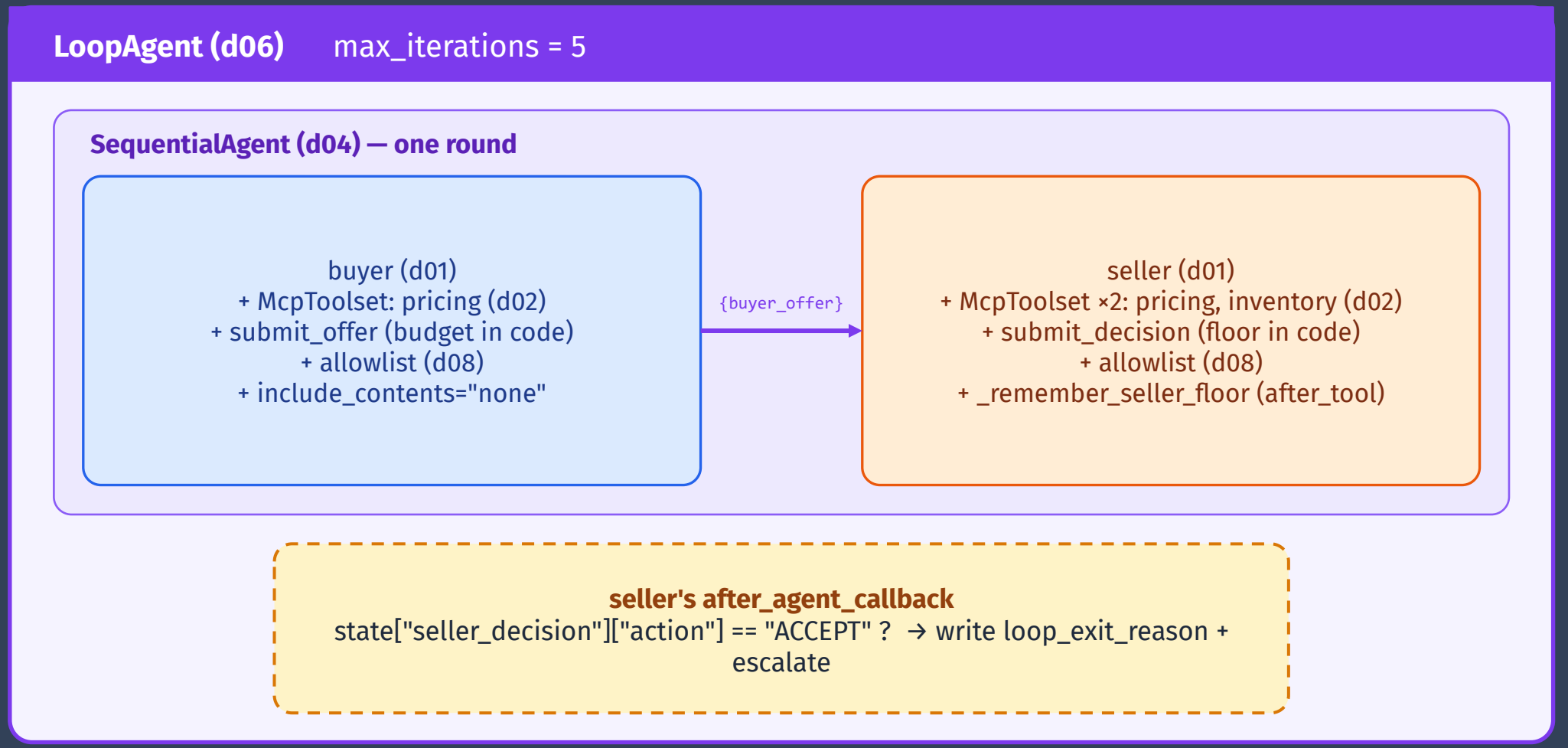
```
def submit_decision(action: str, price: int, tool_context: ToolContext) -> dict:
    decision = _apply_floor_policy(action.upper(), price,                # floor enforced in code
        buyer_offer=tool_context.state.get("buyer_offer_price"), floor=tool_context.state.get("seller_floor_price"))
    tool_context.state["seller_decision"] = {"action": decision["action"], "price": decision["price"]}
    return {"recorded": decision["action"], "price": decision["price"], "note": decision["note"]}

def _check_agreement(callback_context):
    decision = callback_context.state.get("seller_decision")
    if isinstance(decision, dict) and decision.get("action") == "ACCEPT":
        callback_context.state["loop_exit_reason"] = f"ACCEPT at ${decision['price']:,}"   # required
        callback_context.actions.escalate = True

buyer = LlmAgent(output_key="buyer_offer", include_contents="none",
    tools=[McpToolset(pricing), submit_offer], before_tool_callback=_enforce_buyer_allowlist, ...)
seller = LlmAgent(output_key="seller_response",
    tools=[McpToolset(pricing), McpToolset(inventory), submit_decision],
    after_tool_callback=_remember_seller_floor, after_agent_callback=_check_agreement, ...)
root_agent = LoopAgent(sub_agents=[SequentialAgent(sub_agents=[buyer, seller])], max_iterations=5)
```

Concept	Where
LlmAgent (d01) + McpToolset (d02)	buyer, seller
output_key + {placeholder} (d04)	buyer writes → seller reads
SequentialAgent + LoopAgent (d04, d06)	one round, max 5 rounds
Callbacks (d08) + submit_offer / submit_decision	allowlists, floor memory, structured signal

Diagram: How It All Connects



ACCEPT → loop exits · COUNTER → next round · 5 rounds → stop

{ik} INTERVIEW KICKSTART

Negotiation: Live Run Results

What actually happened (actual run, gpt-4o)

Round	Buyer	Seller	Recorded
1	get_market_price + calculate_discount → submit_offer(425000)	3 MCP lookups (floor saved to state) → submit_decision(COUNTER, 477000)	COUNTER $477K
2	submit_offer(438000)	submit_decision(COUNTER, 470000)	COUNTER $470K
3	submit_offer(447000)	submit_decision(ACCEPT, 447000) → loop_exit_reason + escalate	ACCEPT $447K

- **Both sides decide through tools:** submit_offer (rejects anything over $460K) and submit_decision (enforces the floor) — offer_history is built from those calls, not from text
- **The loop ends on the ACCEPT round:** the seller runs last, its after_agent_callback escalates with a state write — no post-acceptance chatter
- **The buyer can't read the seller's floor** (include_contents="none"), so its offers climb 2–4% a round instead of jumping to $445K

Prices vary run to run; across three recent runs the deal closed between $446K and $453K.

Getting Results: Polling, Streaming, Push

Mechanism	How	Needs capability	Connection	In this workshop
Polling	message/send with blocking: false → task comes back at once; then tasks/get until final	none	A short request per check	Demo 13 — the buyer
Streaming	message/stream → one open response; each update is an SSE event	streaming	One long-lived connection	Demo 13 — the seller
Push	message/send with a webhook → the server POSTs each update to it	pushNotifications	None held open	Not demoed (see below)

```
if card.capabilities.streaming:     # -> message/stream
    config = ClientConfig(httpx_client=http, streaming=True)
else:                               # -> message/send + tasks/get
    config = ClientConfig(httpx_client=http, streaming=False,
                          polling=True)
client = ClientFactory(config).create(card)
```

- **Rule of thumb:** seconds → block or poll; want progress → stream; minutes, or the client may go away → push
- **Why push isn't demoed:** ADK's built-in A2A server stores a client's webhook but never calls it (no push sender)

Demo 13: A2A Orchestrated Negotiation

```
adk web --a2a m2_adk_multiagents/negotiation_agents/                    # terminal 1
python m2_adk_multiagents/a2a_demos/a2a_13_orchestrated_negotiation.py   # terminal 2
```

```
Buyer  Capabilities: streaming=False  -> use message/send + tasks/get polling
Seller Capabilities: streaming=True   -> use message/stream
ROUND 1
    [buyer <- poll] task 375af661 submitted - polling tasks/get
    [buyer] task 375af661 completed after 6 polls
    [seller -> stream] working: calling get_market_price, get_inventory_level, get_minimum_acceptable_price
    [seller -> stream] working: calling submit_decision
    [seller -> stream] artifact received
ROUND 2 ... ROUND 3
SELLER: I will ACCEPT your offer of $445,000.        DEAL REACHED in round 3!
A2A tasks created: 6 (one per message)
```

- **The card decides the mechanism:** the script never hard-codes "poll the buyer" — it reads card.capabilities
- **Separate contextIds per agent:** the script is the matchmaker; neither agent knows about the other
- **A task that doesn't complete stops the negotiation** — the script checks task.status.state (demo 14)

{ik} | INTERVIEW KICKSTART

Demo 14: Error Handling (Concept)

```
python m2_adk_multiagents/a2a_demos/a2a_14_error_handling.py
```

Things go wrong at four layers — and only two of them raise

Layer	How it shows up in the client	Examples in the demo
1. Transport	Exception: A2AClientHTTPError (status_code), A2AClientTimeoutError	Agent not running (503), unknown agent path (404), request timed out
2. Protocol	Exception: A2AClientJSONRPCError (.error.code) — HTTP 200 on the wire	Task not found (-32001), not cancelable (-32002), capability not offered, bad request (-32601 / -32602)
3. Task	NO exception — task.status.state == "failed", reason in status.message	A FilePart (application/zip) the model can't read
4. Application	NO exception — state "completed", but the agent didn't do it	An offer with no price: seller asks to clarify, records no decision

Layers 3 and 4 are the ones people miss: the call "succeeds", so you have to look at task.status.state — and then at the content.

{ik} | INTERVIEW KICKSTART

Demo 14: Results

Actual run (ADK 2.11, gpt-4o, abridged)

```
Agent Card, nothing running      A2AClientHTTPError  status=503  (retried with backoff, then gave up)
Agent Card, unknown agent         A2AClientHTTPError  status=404  (fail fast)
Blocking send, 2s timeout         A2AClientTimeoutError  -> the server may STILL be working; no taskId to check
Recovery: blocking=false + poll   task submitted at once -> completed
tasks/get unknown id              A2AClientJSONRPCError code=-32001 'Task not found'
tasks/cancel completed task       A2AClientJSONRPCError code=-32002 'Task cannot be canceled'
tasks/cancel running task         no error - task is now canceled
push config on buyer              code=-32603 'Push notifications are not supported by the agent'
raw message/stream to buyer       HTTP 200 {"error": {"code": -32603, "message": "Streaming is not supported..."}}
FilePart application/zip          no exception. state=failed  "LiteLlm(BaseLlm) does not support ... application/zip"
offer with no price               no exception. state=completed  submit_decision NOT called -> ask again
```

Layer	Detect with	Retry?
Transport	A2AClientHTTPError / TimeoutError	Connection, 5xx: yes, with backoff. Timeout: prefer blocking=false + tasks/get. 4xx: fix the request
Protocol	A2AClientJSONRPCError .error.code	No — fix the request, or use what the card offers
Task / Application	task.status.state, then the content	Not with the same input; read status.message / ask again

{ik} INTERVIEW KICKSTART

Module 2 - Implementation Notes: Google ADK + A2A Protocol



{ik} | INTERVIEW KICKSTART

No Question Is Too Small Every Question Deserves an Answer!!

Ask about anything:

- Concepts from today's modules
- How this applies to your work
- Implementation details
- Clarifications on what we covered

Quick Request

- Please do not mind if a question gets skipped accidentally
- Repost politely in chat if needed
- We will try our best to cover as many questions as possible

Remember: If you're wondering about it, others probably are too.





{ik} | INTERVIEW KICKSTART